REPUBLIQUE DU CAMEROUN
PAIX - TRAVAIL – PATRIE
****
UNIVERSITE DE DSCHANG
****
ÉCOLE DOCTORALE

REPUBLIC OF CAMEROON
PEACE - WORK - FATHERLAND
****
UNIVERSITY OF DSCHANG
****
POST GRADUATE SCHOOL

DSCHANG SCHOOL OF SCIENCES AND TECHNOLOGY

UNITE DE RECHERCHE EN INFORMATIQUE FONDAMENTALE, INGENIERIE ET APPLICATIONS (URIFIA)

SUJET :

ARCHITECTURE ET MÉTHODES D'UN FRAMEWORK DE CRÉATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES BPMN

Mémoire soutenu publiquement en vue de l'obtention du

Diplôme de Master en Informatique

Option : Informatique Fondamentale
Spécialité : Intelligence Artificielle

Par :

NGONG KOUT YURI ROOSEVELT

Matricule : ***CM-UDS-24SCI1029***

Licence en Informatique Fondamentale

Sous la direction de :

ZEKENG NDADJI Milliam Maxime
Chargé de Cours, Université de Dschang

Année académique 2025/2026

DÉCLARATION D'AUTHENTICITÉ

Je, soussigné NGONG KOUT Yuri Roosevelt, étudiant au Département de Mathématiques et Informatique, Faculté des Sciences, Université de Dschang, certifie que ce mémoire de Master intitulé « **Architecture et méthodes d'un framework de création d'applications web pilotées par des modèles BPMN** », réalisé au sein de l'Unité de Recherche en Informatique Fondamentale, Ingénierie et Applications (URIFIA) sous la supervision du Dr ZEKENG NDADJI Milliam Maxime, est original. Il n'a jamais été publié auparavant pour l'obtention d'un quelconque diplôme.

L'étudiant
NGONG KOUT Yuri Roosevelt

.....

Le directeur de mémoire
ZEKENG NDADJI Milliam Maxime
Chargé de Cours, Université de Dschang

.....

Le chef de département

.....

DÉDICACES

À Monsieur **KOUT MBEPA GEORGES** et Madame
NGUIONG NGONG ALINE PRISCILLE, pour leur amour,
leur soutien et leurs sacrifices.

REMERCIEMENTS

Tout d'abord, je remercie infiniment *Dieu Tout-Puissant* pour ses bénédictions, sa grâce et son infinie bonté. Je tiens ensuite à adresser mes sincères remerciements :

— au **Dr ZEKENG NDADJI Milliam Maxime**, mon encadreur académique, pour la confiance qu'il m'a témoignée en acceptant la direction scientifique de mes travaux. Je lui suis reconnaissant de m'avoir fait bénéficier, tout au long de cette recherche, de sa rigueur intellectuelle, de sa disponibilité, de sa bienveillance, de son inspiration constante et de ses précieux conseils. Son expertise et son soutien indéfectible ont été essentiels à l'élaboration de ce mémoire ;

— à tous les **enseignants du Département de Mathématiques et Informatique de la Faculté des Sciences de l'Université de Dschang**, pour la qualité de leurs enseignements et leurs précieux conseils ;

— aux distingués membres du jury qui ont accepté d'examiner ce travail. Leurs observations contribueront à enrichir ma formation ;

— à mes frères et sœurs, **FOSSI Benson, MBEPA Nathanaël, MOLLO Josue, KOUT Sterve, GOUA Dorcas, MANGHA Déograce et KENGNE Godwin**, pour leur soutien inconditionnel et leur affection ;

— aux familles **MBEPA et NGONG**, pour leur soutien et leurs conseils ;

— à mes précieux amis, **Aubin, Triomphant, Marie, Vergez, Jaures, Fomo, Dorine,Francelle, Dominique, Warren et Souleymane**, ainsi qu'à tous ceux que je n'ai pas pu citer et qui ont partagé avec moi cette aventure académique ;

— à mes aînés académiques, **Daniela et Dimitri**, pour leur soutien inestimable et leurs conseils éclairés, qui ont enrichi mon parcours et nourri ma quête de connaissance ;

— Que tous ceux dont les noms ne figurent pas sur cette page ne se sentent pas lésés : leur contribution m'est également précieuse.

TABLE DES MATIÈRES

Dédicaces ii
Remerciements iii
Table des Matières iv
List of Symbols viii
Liste des acronymes ix
Liste des tableaux x
Table des figures xi
Résumé xiii
Abstract xiv

Introduction Générale **1**
Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 4

Chapitre I — — **Généralités sur le Business Process Model and Notation et applications web métier** **5**
I.1 - Introduction 5
I.2 - Processus métier et gestion des processus métier (BPM) 5
I.2.1 - Processus métier 6
I.2.2 - Gestion des processus métier (BPM) 7
I.2.2.1 - Définitions de la gestion des processus métier 7
I.2.3 - Cycle de vie du BPM 7
I.2.3.1 - Identification du processus *(Process Identification)* . . 8
I.2.3.2 - Découverte des processus *(Process Discovery)* 8
I.2.3.3 - Analyse du processus *(Process Analysis)* 8
I.2.3.4 - Refonte du processus *(Process Redesign)* 8
I.2.3.5 - Mise en œuvre du processus *(Process Implementation)* 8
I.2.3.6 - Suivi du processus *(Process Monitoring)* 9
I.2.4 - Systèmes de gestion des processus métier 9
I.3 - Modélisation des processus métier 10
I.3.1 - Business Process Model and Notation (BPMN) 10
I.3.1.1 - Définition et objectif 10

I.3.1.2 - Historique du BPMN 11
I.3.1.3 - Éléments de base de BPMN 11
I.3.1.4 - Types de diagrammes BPMN 16
I.4 - Ingénierie dirigée par les modèles 17
I.4.1 - Principe général de l'ingénierie dirigée par les modèles 17
I.4.2 - Notions de base en ingénierie dirigée par les modèles 18
I.4.3 - Transformation de modèles 18
I.4.4 - Model-Driven Architecture 19
I.5 - Applications web métier 20
I.5.1 - Généralités sur le développement web 20
I.5.1.1 - Historique du développement web 20
I.5.1.2 - Définition d'application web et concepts du web . . . 21
I.5.1.3 - Frameworks d'application web 22
I.5.1.4 - Sécurité dans les applications web 23
I.5.2 - Applications web métier 24
I.5.2.1 - Définitions et principe 24
I.5.2.2 - Moteur de processus 24
I.6 - Conclusion 25

Chapitre II —— **État de l'art sur la génération d'applications web pilotées par des modèles** **26**
II.1 - Introduction 26
II.2 - Démarche d'analyse des travaux existants 26
II.3 - Revue de la littérature 27
II.3.1 - Systèmes d'information orientés processus et plateformes d'exécution 27
II.3.1.1 - Moteurs BPMS et plateformes d'exécution 28
II.3.1.2 - Architecture web pilotée par BPMN 28
II.3.1.3 - WebWorkFlow 29
II.3.1.4 - Plateforme AKIP 30
II.3.2 - Dérivation d'artefacts applicatifs à partir de BPMN 30
II.3.2.1 - Transformation vers des modèles d'interaction 31
II.3.2.2 - Génération de prototypes et d'API 32
II.3.2.3 - Vers une génération intégrée des artefacts applicatifs 32
II.3.3 - Ingénierie dirigée par les modèles et génération de systèmes orientés processus 34
II.3.3.1 - Composition de langages et génération progressive . 34

II.3.3.2 - Gabarits et compléments aux modèles BPMN 35
II.3.4 - Maintenabilité du code généré et mise en perspective avec le low-code 36
II.3.4.1 - Gestion conjointe du code généré et du code manuel 36
II.3.4.2 - Mise en perspective avec les approches low-code . . 37
II.4 - Discussion et positionnement de la recherche 37
II.5 - Conclusion 40

Chapitre III — — **Approche d'ingénierie proposée pour la génération d'applications web pilotées par BPMN** **42**
III.1 - Introduction 42
III.2 - Vue d'ensemble de l'approche 43
III.2.1 - Problème traité et principe général 43
III.2.2 - Positionnement dans le cycle MDA 43
III.2.3 - Contribution méthodologique de l'approche 44
III.3 - Architecture globale du framework proposé 44
III.3.1 - Modèles d'entrée de ProOWeb 45
III.3.2 - Noyau de génération du framework 45
III.3.3 - Application web générée 46
III.3.4 - Exploitation et évolution de l'application 46
III.4 - Modèle d'entrée enrichi : spécification spec-v1 46
III.4.1 - Grammaire de spec-v1 47
III.4.2 - Propriétés de la grammaire spec-v1 49
III.4.3 - Métadonnées des activités 49
III.4.4 - Affectation des tâches et sécurité 50
III.4.5 - Entrées, sorties et données partagées 51
III.4.6 - Règles de visibilité 52
III.5 - Règles de transformation et génération de l'application web 52
III.5.1 - Validation avant génération 52
III.5.2 - Transformation modèle-à-modèle 53
III.5.2.1 - Algorithme de transformation M2M 53
III.5.2.2 - Propriétés de l'algorithme de transformation M2M . 55
III.5.3 - Transformation modèle-à-texte 56
III.5.3.1 - Algorithme de génération Model-to-Text 56
III.5.3.2 - Propriétés de l'algorithme de transformation M2T . 58
III.5.4 - Règles de projection vers les artefacts 59
III.5.5 - Traçabilité de bout en bout 59

III.6 - Préservation du code personnalisé et synchronisation 60
III.6.1 - Problème de la régénération 60
III.6.2 - Manifeste et contrat de propriété 61
III.6.3 - Préservation côté logique applicative 61
III.6.4 - Préservation côté interface utilisateur 61
III.6.5 - Cycle de synchronisation itératif 62
III.7 - Conclusion 62

Chapitre IV — **Mise en œuvre et évaluation de ProOWeb** **64**
IV.1 - Introduction 64
IV.2 - Environnement de mise en œuvre 65
IV.2.1 - Objectif de réalisation 65
IV.2.2 - Choix technologiques 65
IV.2.3 - Chaîne de génération 66
IV.3 - Cas d'étude organisationnel 67
IV.3.1 - Présentation de l'organisation retenue 67
IV.3.2 - Acteurs et responsabilités 67
IV.3.3 - Processus retenus 68
IV.3.4 - Scénario nominal et scénarios alternatifs 69
IV.4 - Déroulement complet du processus de commande client 69
IV.4.1 - Initialisation du projet 70
IV.4.2 - Modélisation du processus 70
IV.4.3 - Enrichissement du BPMN avec *spec-v1* 71
IV.4.4 - Analyse et validation 72
IV.4.5 - Construction du modèle pivot 72
IV.4.6 - Génération des artefacts applicatifs 74
IV.4.7 - Fonctionnement dans l'application web générée 75
IV.5 - Discussion et positionnement 78
IV.6 - Conclusion 79

Conclusion générale **80**
Bilan du travail 80
Quelques perspectives 81

Bibliographie **83**

LIST OF SYMBOLS

G_{spec}	modèle grammatical attribué de *spec-v1* ;
p	identifiant du processus BPMN enrichi ;
v	version de la spécification ;
R	ensemble des rôles déclarés ;
$S = (R_s, A_s, b_s)$	configuration du démarrage du processus ;
R_s	rôles autorisés à démarrer une instance ;
A_s	activités initiales du processus ;
b_s	autorisation de démarrage automatique sans entrée manuelle ;
M	rôles autorisés à superviser le processus ;
$A = \{a_1, \ldots, a_n\}$	ensemble des activités enrichies ;
$D = \{d_1, \ldots, d_m\}$	ensemble des entités de données partagées ;
C	ensemble des contraintes de validité ;
a_i	activité enrichie d'indice i ;
id_i	identifiant BPMN de l'activité a_i ;
t_i	type de l'activité a_i ;
R_i	rôles candidats de l'activité a_i ;
α_i	politique d'affectation de l'activité a_i ;
I_i	sources d'entrée de l'activité a_i ;
$O_i = (s_i, \mu_i)$	sortie, stratégie de stockage et mappings de l'activité ;
$V_i = (R_i^a, R_i^d)$	visibilité de l'activité et de ses données ;
X_i	paramètres d'exécution d'une activité automatique ;
$d_j = (k_j, F_j)$	entité partagée, sa clé et son ensemble de champs.

LISTE DES ACRONYMES

AFNOR	Association française de normalisation ;
AKIP	AKIP Process Automation Platform ;
API	Application Programming Interface ;
ATL	ATLAS Transformation Language ;
BPM	Business Process Management ;
BPMI	Business Process Management Institute ;
BPMN	Business Process Model and Notation ;
BPMS	Business Process Management System ;
CIM	Computation Independent Model ;
DMN	Decision Model and Notation ;
EBNF	Extended Backus–Naur Form ;
EPC	Event-driven Process Chain ;
HTML	HyperText Markup Language ;
HTTP	HyperText Transfer Protocol ;
IBM	International Business Machines ;
IDM	Ingénierie dirigée par les modèles ;
IFML	Interaction Flow Modeling Language ;
jBPM	Java Business Process Management ;
JSON	JavaScript Object Notation ;
M2M	Model-to-Model ;
M2T	Model-to-Text ;
MDA	Model Driven Architecture ;
MOF	Meta-Object Facility ;
OMG	Object Management Group ;
PIM	Platform Independent Model ;
PSM	Platform Specific Model ;
QVT	Query/View/Transformation ;
RBAC	Role-Based Access Control ;
REST	Representational State Transfer ;
RFC	Request for Comments ;
SPA	Single Page Application ;
SWEL	Domain-Specific Language for Modeling Data-Intensive Workflows ;
UML	Unified Modeling Language ;
WebDSL	Web Domain-Specific Language ;
XOR	Exclusive OR.

LISTE DES TABLEAUX

I - Composantes essentielles d'un processus métier 6

II - Grille concrète de lecture des travaux étudiés 27
III - Récapitulatif des travaux étudiés 38
IV - Limites transversales des travaux étudiés et implications pour la recherche 40

V - Positionnement de ProOWeb dans le cycle MDA 43
VI - Propriétés de la grammaire spec-v1 49
VII - Stratégies d'affectation 50
VIII - Entrées, sorties et données partagées d'une activité BPMN enrichie . 51
IX - Contrôles réalisés avant génération 53
X - Propriétés de l'algorithme de transformation M2M 55
XI - Structures principales du modèle pivot ProOWeb 56
XII - Propriétés de l'algorithme de transformation M2T 58
XIII - Règles de projection vers l'application générée 59
XIV - Traçabilité d'une activité BPMN vers les artefacts générés 60
XV - Mécanismes de préservation et de synchronisation dans ProOWeb . . 62

XVI - Principaux choix de mise en œuvre de ProOWeb 66
XVII - Acteurs du cas d'étude e-commerce 68
XVIII - Processus métier retenus dans le cas d'étude 69

TABLE DES FIGURES

1 - Exemple de processus métier 6
2 - Cycle de vie du BPM [12] 7
3 - Historique du BPMN [15] 11
4 - Les différents éléments de BPMN 2.0. 12
5 - Les types des tâches dans BPMN 2.0. 13
6 - Événements dans BPMN 2.0. 13
7 - Les branchements de BPMN 2.0. 14
8 - Les flux de séquence de BPMN 2.0. 14
9 - La représentation d'un pool avec deux lanes en BPMN 2.0. 15
10 - Les données de BPMN 2.0. 15
11 - Les artéfacts. 16
12 - Exemple de diagramme de processus 16
13 - Exemple de diagramme de collaboration 17
14 - Exemple de diagramme de chorégraphie 17
15 - Pyramide de modélisation de l'OMG, d'après [5]. 18
16 - Schéma de base d'une transformation de modèle [16]. 19
17 - Illustration des trois niveaux d'abstraction de la MDA [16]. 20
18 - Cœur du modèle RBAC, adapté de [25] 24
19 - Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [26]. 28
20 - Fonctionnalités et technologies d'une application web pilotée par processus [33]. 29
21 - Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [34]. 29
22 - Architecture et principe de génération d'une application dans AKIP, adaptés de [35]. 30
23 - Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [36]. 31
24 - Cycle de prototypage rapide d'applications web proposé par [39]. . . . 33
25 - Extraits de code généré pour le contrôleur et la vue dans l'approche de [39]. 33

26 - Génération en deux étapes d'un système d'information orienté processus, adaptée de [9]. 34

27 - Architecture générale de ProOWeb 45
28 - Algorithme de transformation modèle-à-modèle BPMN/spec-v1 vers le modèle pivot 54
29 - Algorithme de transformation Model-to-Text vers le socle applicatif . 57

30 - Chaîne de génération de ProOWeb 67
31 - Initialisation du projet `E-commerce_Application` 70
32 - Modélisation du processus de commande client dans ProOWeb 71
33 - Spécification ajoutée à l'activité « Passer la commande » 71
34 - Accès à la liste des processus et démarrage de la commande client . . 75
35 - Formulaire associé à l'activité « Passer la commande » 76
36 - Transition vers l'activité de vérification affectée au Service commercial 77
37 - Suivi de l'instance de commande 77

RÉSUMÉ

Ce mémoire s'intéresse à la génération d'applications web métier à partir de modèles de processus. Il part du constat que, dans de nombreuses organisations, les processus décrivent déjà une part essentielle du travail à informatiser : les activités à enchaîner, les acteurs concernés, les informations manipulées et les décisions à prendre. Pourtant, au moment de développer une application web, ces modèles restent souvent cantonnés à un rôle documentaire et leur contenu doit être réinterprété manuellement par les développeurs. Cette séparation fragilise la cohérence entre le besoin métier et le logiciel produit, complique la maintenance et rend l'évolution des applications plus coûteuse lorsque les procédures changent. L'objectif de ce travail est donc de concevoir un framework capable d'exploiter plus directement un modèle BPMN enrichi pour produire une application web métier structurée, traçable et adaptable. Pour cela, nous avons conçu ProOWeb, un framework qui associe le modèle BPMN, chargé de représenter le déroulement du processus, à une spécification complémentaire. Cette spécification apporte les informations que le modèle ne porte pas suffisamment, notamment les rôles, les affectations, les données, les entrées, les sorties, les règles de visibilité et les indications nécessaires à la génération des formulaires. Le framework valide ces éléments, les organise dans un modèle pivot, puis les projette vers les artefacts d'une application web métier : vues de traitement, logique applicative, structures de données, règles d'accès, suivi des instances et traces de génération. Le travail met également l'accent sur la régénération contrôlée, afin que l'application produite puisse évoluer sans rompre le lien entre le processus, la spécification et le code. Ainsi, ProOWeb ne vise pas seulement à automatiser l'écriture de fichiers, mais à rapprocher durablement la modélisation métier et la réalisation logicielle.

Mots clés : Processus métier, application web métier, BPMN, ProOWeb, génération d'application, ingénierie dirigée par les modèles, spécification enrichie, régénération contrôlée.

ABSTRACT

This thesis focuses on generating business web applications from process models. It begins with the observation that, in many organizations, processes already describe a significant portion of the work to be computerized : the activities to be sequenced, the stakeholders involved, the information handled, and the decisions to be made. However, when developing a web application, these models often remain confined to a purely documentary role, and their content must be manually reinterpreted by developers. This separation weakens the coherence between business needs and the software produced, complicates maintenance, and makes application evolution more expensive when procedures change. The objective of this work is therefore to design a framework capable of making more direct use of an enriched BPMN model to produce a structured, traceable, and adaptable business web application. To this end, we have designed ProOWeb, a framework that combines the BPMN model, responsible for representing the process flow, with a complementary specification. This specification provides the information that the model does not adequately contain, including roles, assignments, data, inputs, outputs, visibility rules, and the necessary information for form generation. The framework validates these elements, organizes them into a pivot model, and then projects them onto the artifacts of a business web application : processing views, application logic, data structures, access rules, instance tracking, and generation traces. The work also emphasizes controlled regeneration, ensuring that the resulting application can evolve without breaking the link between the process, the specification, and the code. Thus, ProOWeb aims not only to automate file writing but also to foster a lasting connection between business modeling and software development.

Keywords : Business process, business web application, BPMN, ProOWeb, application generation, model-driven engineering, rich specification, controlled regeneration.

INTRODUCTION GÉNÉRALE

SOMMAIRE

Contexte du travail 1
Problématique et objectifs 2
Organisation du manuscrit 4

Contexte du travail

Une part importante des services proposés aujourd'hui sur le web repose sur des activités organisées sous forme de processus métier. Lorsqu'un usager effectue une inscription en ligne, dépose une demande administrative, suit une réclamation, réalise une commande ou valide une opération interne dans une organisation, il participe à une suite d'actions coordonnées, portées à la fois par des humains, des règles métier et des systèmes logiciels. Dans ce contexte, l'application web ne se limite plus à présenter des pages ou à stocker des informations : elle devient le support d'exécution d'un travail métier, avec des rôles, des formulaires, des données, des validations et des enchaînements d'activités.

Le Business Process Management (BPM) fournit un cadre pour identifier, modéliser, analyser et améliorer ces processus. Les travaux de référence dans ce domaine montrent que la maîtrise d'un processus passe autant par sa représentation explicite que par sa capacité à être exécuté et adapté dans le système d'information [1, 2]. Parmi les notations utilisées, BPMN occupe une place importante, car elle permet de décrire les acteurs, les événements, les tâches et les flux de contrôle d'un processus selon une notation largement reconnue [3]. Cependant, un modèle BPMN, même correct du point de vue du processus, ne suffit généralement pas à produire directement une application web métier complète. Il indique ce qui doit s'enchaîner, mais ne décrit pas toujours avec précision les écrans attendus, les données manipulées, les champs visibles ou obligatoires, les règles de création des

formulaires, ni la manière dont les utilisateurs interagissent concrètement avec le processus.

Parallèlement, l'ingénierie dirigée par les modèles propose de placer les modèles au centre du développement logiciel, afin de transformer des descriptions abstraites en artefacts plus proches de l'exécution [4, 5]. Cette orientation est particulièrement intéressante pour les applications web orientées processus, car elle permet d'envisager une continuité entre la compréhension métier, la spécification du processus et la production de l'application. Toutefois, les approches existantes restent souvent limitées à la génération de fragments d'interface, de prototypes ou de structures partielles. Plusieurs travaux ont exploré le passage de BPMN vers des modèles d'interaction ou vers des applications guidées par les processus [6–9], mais le problème demeure lorsqu'il s'agit de produire une application web métier fonctionnelle, utilisable et cohérente avec les exigences exprimées dans le modèle.

C'est dans cette perspective que s'inscrit ce mémoire, intitulé « Architecture et méthodes d'un framework de création d'applications web pilotées par des modèles BPMN ». Le travail présenté porte sur ProOWeb, un framework d'ingénierie visant à générer un socle d'application web métier fonctionnel à partir de modèles BPMN enrichis. L'idée centrale n'est donc pas seulement de dessiner un processus ni de produire une maquette, mais de faire du modèle enrichi une source exploitable pour obtenir une application capable de gérer les acteurs, les tâches, les données, les formulaires, les transitions et le suivi des instances. ProOWeb cherche ainsi à rapprocher la modélisation métier et la réalisation logicielle, tout en conservant une organisation suffisamment claire pour permettre l'évolution de l'application générée.

Problématique et objectifs

La difficulté principale vient du décalage entre la richesse attendue d'une application web métier et les informations habituellement contenues dans un modèle BPMN standard. Un processus peut préciser qu'une tâche doit être exécutée par un acteur donné, mais l'application correspondante doit aussi savoir quelles données afficher, quelles informations demander, quelles validations appliquer, quelles pages produire, comment relier les actions de l'utilisateur à l'avancement du processus et comment préserver les parties personnalisées lorsque le modèle évolue. Sans mécanisme structuré, le passage du modèle au code devient manuel, répétitif et fragile ; il expose le projet à des incohérences entre le processus conçu et l'application réellement livrée.

La problématique de ce travail peut donc être formulée de la manière suivante : comment concevoir un framework permettant, à partir de modèles BPMN enrichis, de générer un socle applicatif web métier fonctionnel, traçable et extensible, tout en conservant la cohérence entre les processus, les données, les rôles, les interfaces et l'évolution future de l'application ?

L'objectif principal de ce mémoire est de concevoir et de mettre à l'épreuve ProOWeb, un framework de génération d'applications web métier pilotées par des modèles BPMN enrichis. La réponse proposée à la problématique repose sur une chaîne explicite : le BPMN décrit le déroulement du travail, *spec-v1* apporte les informations applicatives manquantes, une validation croisée contrôle leur cohérence, puis des contrats intermédiaires stabilisent les transformations modèle-à-modèle et modèle-à-texte. Ces transformations produisent un socle backend/frontend, les structures de données, les règles de sécurité et les ressources du runtime. Un manifeste relie enfin les sources aux fichiers générés et prépare leur régénération contrôlée.

De manière spécifique, il s'agit de :

— étudier les notions de processus métier, de BPMN, d'applications web orientées processus et d'ingénierie dirigée par les modèles, afin de poser les bases conceptuelles du travail ;
— analyser les travaux existants sur la génération d'applications web à partir de modèles de processus, en mettant en évidence leurs apports et leurs limites ;
— définir et formaliser un modèle d'entrée enrichi associant BPMN et *spec-v1*, ainsi que les contrôles nécessaires à leur mise en cohérence ;
— proposer l'architecture de ProOWeb, ses contrats intermédiaires, les algorithmes de transformation et ses mécanismes de traçabilité et de régénération ;
— mettre en œuvre cette chaîne et l'évaluer, dans les limites d'un cas d'étude, sur trois processus d'une même organisation.

La contribution ne se réduit donc pas à un outil de production de fichiers. Elle porte sur une démarche d'ingénierie qui précise les informations attendues, contrôle les modèles d'entrée, organise leur transformation et conserve la trace des décisions projetées dans le code. ProOWeb vise ainsi un socle applicatif fonctionnel, traçable et extensible, que le développeur doit encore vérifier et adapter aux contraintes propres à son organisation.

Organisation du manuscrit

La suite de ce mémoire est organisée en quatre chapitres, suivis d'une conclusion générale.

Chapitre I : Généralités sur le Business Process Model and Notation et les applications web métier. Ce chapitre présente les concepts de base nécessaires à la compréhension du sujet. Il revient sur les processus métier, le BPM, la notation BPMN, l'ingénierie dirigée par les modèles et les applications web métier. Il permet ainsi de fixer le vocabulaire et le cadre conceptuel utilisés dans la suite du mémoire.

Chapitre II : Revue de la littérature sur la génération d'applications web pilotées par des modèles. Ce chapitre étudie les travaux existants liés à l'automatisation des processus, aux systèmes BPM, à la génération de code, aux approches low-code/no-code et aux transformations de modèles. Il met en évidence les limites rencontrées lorsqu'il faut passer d'un modèle de processus à une application web métier complète, fonctionnelle et évolutive.

Chapitre III : Framework d'ingénierie proposé pour la génération d'applications web pilotées par BPMN. Ce chapitre constitue le cœur de la contribution. Il présente ProOWeb, son principe général, son architecture, son modèle d'entrée enrichi, ses règles de transformation et les mécanismes prévus pour produire une application web métier fonctionnelle tout en conservant la possibilité d'évolution et de personnalisation.

Chapitre IV : Mise en œuvre et évaluation de ProOWeb. Ce chapitre est consacré à la réalisation concrète du framework et à son évaluation. Il présente l'application générée pour une organisation de commerce électronique, les choix de mise en œuvre, les trois processus retenus, les interfaces obtenues, les résultats observés ainsi que les limites du travail réalisé.

Conclusion générale : elle rappelle la problématique traitée, synthétise les contributions du mémoire, discute les résultats obtenus et ouvre quelques perspectives pour l'amélioration de ProOWeb.

I

CHAPITRE

GÉNÉRALITÉS SUR LE BUSINESS PROCESS MODEL AND NOTATION ET APPLICATIONS WEB MÉTIER

SOMMAIRE

I.1 - Introduction 5
I.2 - Processus métier et gestion des processus métier (BPM) 5
I.3 - Modélisation des processus métier 10
I.4 - Ingénierie dirigée par les modèles 17
I.5 - Applications web métier 20
I.6 - Conclusion 25

I.1. Introduction

Dans ce chapitre, nous présentons les notions de base nécessaires à la compréhension de notre travail. Nous abordons dans un premier temps les processus métier, le BPM et BPMN. Dans un second temps, nous rappelons les concepts du développement web utiles à l'étude des applications métier. Cette présentation permet de préciser le lien entre modèle métier, moteur de processus et application web.

I.2. Processus métier et gestion des processus métier (BPM)

I.2.1. Processus métier

Un processus peut être compris comme un ensemble d'activités corrélées qui transforment des éléments d'entrée en éléments de sortie. Cette définition, proche de celle de l'ISO 9000, met l'accent sur l'enchaînement, la transformation et le résultat attendu. Hammer et Champy définissent également le processus comme un ensemble finalisé d'activités produisant une valeur pour un client [10].

Un processus métier est un processus lié à un objectif concret de l'organisation. Il décrit comment une structure produit un service, traite une demande, prend une décision ou coordonne plusieurs intervenants.

Le caractère métier du processus est important. Le processus exprime d'abord une réalité organisationnelle avant de devenir une réalité informatique. Il précise ce qui doit être fait, par qui, avec quelles informations, selon quelles règles et avec quel résultat.

Un processus métier comporte généralement trois dimensions. La dimension fonctionnelle répond à la question : que faut-il faire ? La dimension organisationnelle répond à la question : qui le fait ? La dimension informationnelle répond à la question : quelles données sont nécessaires ou produites ? Ces trois dimensions forment une base utile pour passer ensuite vers la conception d'un système d'information.

TABLE I – *Composantes essentielles d'un processus métier*

Composante	Rôle dans le processus
Entrées	Éléments nécessaires au démarrage ou à l'exécution : demande, formulaire, document, événement ou information reçue.
Activités	Opérations réalisées par des acteurs humains ou des systèmes.
Acteurs	Personnes, services, rôles ou applications responsables des activités.
Règles	Conditions qui orientent l'exécution, les décisions, les validations et les autorisations.
Sorties	Résultats produits : dossier validé, demande rejetée, document généré, notification envoyée.

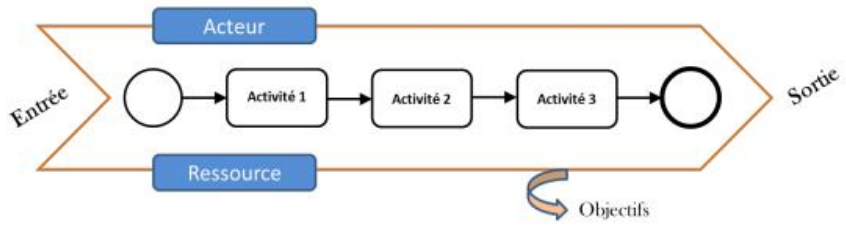


FIGURE 1 – *Exemple de processus métier*

I.2.2. Gestion des processus métier (BPM)

I.2.2.1. Définitions de la gestion des processus métier

Plusieurs définitions ont été données pour le terme de gestion des processus métier.

D'après van der Aalst et al., le *Business Process Management* (BPM) comprend « les méthodes, les techniques et les outils permettant de concevoir, d'exécuter, de gérer et d'analyser les processus opérationnels, en s'appuyant sur des acteurs pouvant être des êtres humains, des organisations, des applications et d'autres sources d'information » [11].

Le *Business Process Management* (BPM) peut également être défini comme l'art de modéliser, de gérer et d'optimiser les processus métier afin d'améliorer la performance des entreprises. Il nécessite l'utilisation d'outils permettant de modéliser, d'intégrer, de superviser et d'analyser le fonctionnement des processus tout au long de leur cycle de vie, dans une logique d'amélioration continue [1, 2].

I.2.3. Cycle de vie du BPM

Selon Marlon Dumas [12], le BPM peut être considéré comme un cycle continu comprenant les phases suivantes

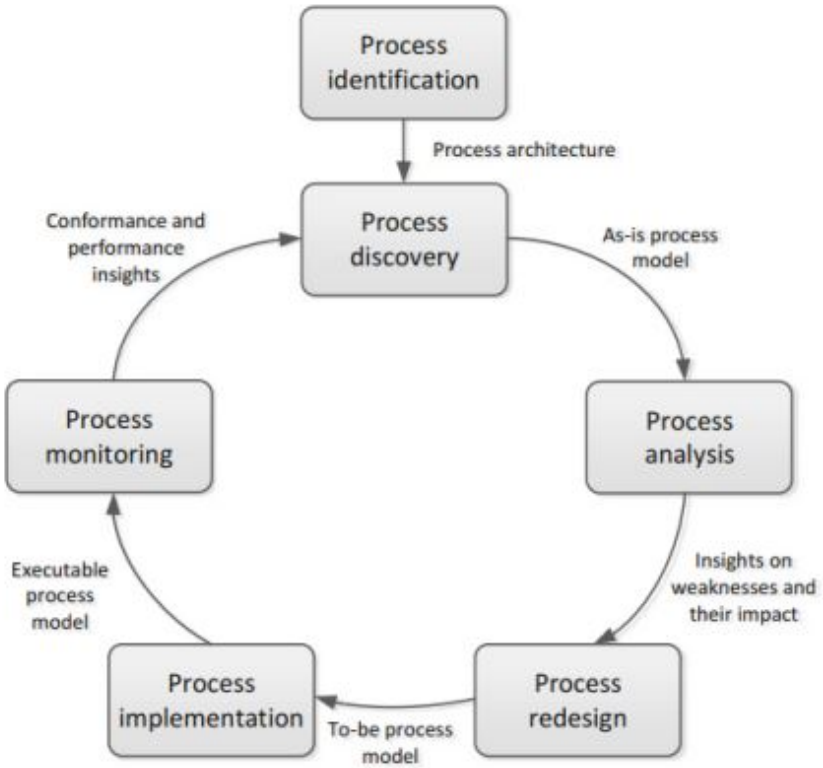


FIGURE 2 – *Cycle de vie du BPM [12]*

I.2.3.1. Identification du processus *(Process Identification)*

Cette étape part d'un problème concret rencontré dans l'entreprise. Il s'agit alors de repérer les processus concernés, d'en préciser les limites et de comprendre les liens qu'ils entretiennent avec les autres processus. Ce travail conduit à une architecture de processus, nouvelle ou actualisée, qui donne une vue d'ensemble du fonctionnement de l'organisation. Cette architecture sert ensuite de base pour choisir le ou les processus qui seront traités dans les autres phases du cycle de vie. L'identification des processus s'accompagne souvent de la définition des indicateurs de performance.

I.2.3.2. Découverte des processus *(Process Discovery)*

Cette phase correspond à la description de la situation existante. Elle consiste à représenter le fonctionnement réel des processus retenus, le plus souvent à travers un ou plusieurs modèles de processus. Elle permet ainsi de disposer d'une vision claire de la manière dont le travail est effectivement réalisé.

I.2.3.3. Analyse du processus *(Process Analysis)*

À ce niveau, les difficultés liées au processus sont relevées, décrites et, lorsque cela est possible, mesurées à l'aide d'indicateurs de performance. Le résultat obtenu est une liste organisée des problèmes constatés. Ces problèmes sont ensuite hiérarchisés selon leur importance, leur impact sur le processus et l'effort nécessaire pour les corriger.

I.2.3.4. Refonte du processus *(Process Redesign)*

Cette phase, aussi appelée amélioration du processus, vise à proposer les modifications capables de résoudre les problèmes identifiés précédemment et d'aider l'organisation à atteindre ses objectifs de performance. Plusieurs possibilités de changement sont étudiées, puis comparées à partir des indicateurs retenus. Ainsi, la refonte reste étroitement liée à l'analyse : chaque solution envisagée doit être évaluée avant d'être retenue. Les options les plus pertinentes sont finalement regroupées pour produire un processus amélioré, généralement présenté sous forme de modèle.

I.2.3.5. Mise en œuvre du processus *(Process Implementation)*

Cette étape consiste à préparer et à appliquer les changements permettant de passer du processus actuel au processus cible. Elle comporte deux dimensions principales : le changement organisationnel et l'automatisation. Le changement organisationnel concerne l'adaptation des pratiques de travail des acteurs impliqués dans

le processus. L'automatisation, quant à elle, renvoie à la mise en place ou à l'amélioration des systèmes informatiques destinés à soutenir le nouveau processus.

I.2.3.6. Suivi du processus *(Process Monitoring)*

Lorsque le processus amélioré est exécuté, des données sont collectées puis analysées afin d'évaluer sa performance par rapport aux objectifs fixés. Cette observation permet de détecter les lenteurs, les erreurs fréquentes ou les écarts par rapport au fonctionnement prévu. Des actions correctives peuvent alors être engagées. De nouveaux problèmes pouvant apparaître dans le même processus ou dans d'autres, le cycle BPM doit être repris de manière continue.

I.2.4. Systèmes de gestion des processus métier

Les systèmes de gestion des processus métier, généralement désignés par l'abréviation BPMS *(Business Process Management Systems)*, désignent des solutions logicielles conçues pour accompagner la mise en œuvre des démarches BPM. Ces environnements peuvent intégrer plusieurs fonctionnalités, notamment la modélisation des processus, l'exécution des workflows, la gestion des tâches, le suivi à travers des tableaux de bord, l'interconnexion avec d'autres applications, la supervision des activités et, dans certains cas, la conception d'interfaces utilisateur [13]. Parmi ces logiciels, nous pouvons citer :

— **Camunda** : Camunda est une plateforme d'orchestration de processus qui s'appuie notamment sur les standards BPMN et DMN. Elle est souvent utilisée pour automatiser des processus complexes impliquant des utilisateurs, des applications, des services et des règles de décision. Camunda est particulièrement adaptée aux environnements techniques distribués et aux architectures orientées services.

— **Bonita / Bonitasoft** : Bonita est une plateforme BPMS orientée vers l'automatisation et l'optimisation des processus métier. Elle permet de modéliser des processus, de les exécuter, de gérer les tâches utilisateurs et d'intégrer les processus avec d'autres systèmes d'information. Son intérêt réside dans son caractère extensible et dans sa séparation entre la modélisation métier, le développement technique et l'exécution des processus.

— **IBM Business Automation Workflow** : IBM Business Automation Workflow est une plateforme d'automatisation des workflows numériques. Elle combine la gestion des processus métier et la gestion de cas afin de coordonner les utilisateurs, les informations et les activités. Elle est surtout

utilisée dans les grandes organisations ayant besoin de solutions robustes, sécurisées et intégrées à leur système d'information.

— **Flowable** : Flowable est une plateforme de gestion et d'automatisation des processus métier basée sur des standards ouverts. Elle est utilisée pour exécuter des workflows, gérer des processus complexes et coordonner des tâches humaines ou automatisées. Son intérêt se trouve dans sa flexibilité et dans sa capacité à être intégrée dans des applications d'entreprise.

I.3. Modélisation des processus métier

La modélisation des processus métier consiste à représenter le fonctionnement d'un processus afin de préciser les activités qui le composent ainsi que leur ordre d'exécution. Dans la littérature, deux grandes approches sont généralement distinguées pour décrire le comportement d'un processus [14] :

— **L'approche impérative** : elle décrit de manière explicite la façon dont le processus doit être exécuté. Elle précise donc l'enchaînement des activités, les conditions de passage d'une étape à une autre et les différents chemins possibles. Les langages tels que BPMN, les diagrammes d'activités UML ou encore les EPC s'inscrivent dans cette logique.

— **L'approche déclarative** : elle s'intéresse davantage aux contraintes que le processus doit respecter qu'à la description détaillée de toutes les séquences d'exécution. Autrement dit, elle indique ce qui est autorisé, obligatoire ou interdit, sans imposer un chemin unique d'exécution. Parmi les langages associés à cette approche, on peut citer Declare, ConDec, PLM-flow et PENELOPE.

I.3.1. Business Process Model and Notation (BPMN)

I.3.1.1. Définition et objectif

BPMN, pour *Business Process Model and Notation*, est un standard de modélisation des processus métier maintenu par l'Object Management Group. Sa version 2.0.2 définit à la fois une notation graphique et des éléments de méta-modèle permettant de représenter des processus, des événements, des activités, des passerelles, des participants et des flux [3]. Son intérêt principal est de proposer un langage visuel compréhensible par les acteurs métier, tout en restant exploitable par des outils informatiques.

BPMN répond à un besoin fréquent dans les organisations : disposer d'une notation commune pour décrire les processus. Avant l'existence d'une notation partagée, chaque équipe pouvait représenter les processus à sa manière, avec des symboles locaux et des règles implicites. Cette diversité rendait les échanges difficiles. BPMN cherche à réduire cette ambiguïté en fournissant une grammaire visuelle plus stable.

I.3.1.2. Historique du BPMN

Von Rosing et al. retracent l'évolution de BPMN comme suit [15] :

— En 2001, il existait de nombreuses notations et plusieurs points de vue sur la modélisation des processus. Les membres de la Business Process Management Initiative (BPMI) ont alors engagé des travaux visant à unifier leur représentation graphique et à rapprocher la notation des langages de processus exécutables.
— En mai 2004, la première version de BPMN, BPMN 1.0, a été publiée.
— Le 6 février 2006, la BPMI (Business Process Management Initiative) a été intégrée à l'OMG qui a depuis maintenu et développé la norme BPMN.
— En janvier 2008, la version BPMN 1.1 a été publiée par l'OMG.
— En 2009, la version BPMN 1.2 a été publiée par l'OMG.
— En janvier 2011, la version BPMN 2.0 a été publiée par l'OMG.
— En décembre 2013, la version 2.0.2 de BPMN a été publiée par l'OMG.



FIGURE 3 – *Historique du BPMN [15]*

I.3.1.3. Éléments de base de BPMN

Un diagramme BPMN est composé d'un ensemble d'éléments graphiques qui permettent de modéliser les activités, les flux, les relations, les données et l'interaction entre ces éléments.

Les différents éléments sont organisés en cinq catégories :

— les objets de flux *(Flow Objects)* ;
— les objets de connexion *(Connecting Objects)* ;
— les partitions *(Swimlanes)* ;

— les données ;
— les artéfacts *(Artifacts)*.

Les différents éléments de BPMN 2.0 sont illustrés dans la figure ci-dessous.

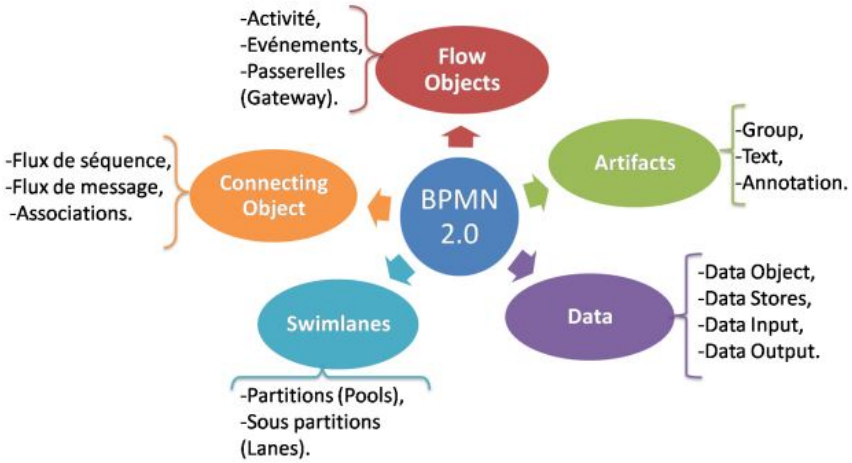


FIGURE 4 – *Les différents éléments de BPMN 2.0.*

A. Les objets de flux

Les objets de flux sont les principaux éléments graphiques qui permettent de définir le comportement d'un processus métier. Il existe trois types d'objets de flux : les activités, les événements et les passerelles.

1. Les activités. Les activités sont représentées par un rectangle aux coins arrondis. Elles correspondent à une action pouvant être réalisée par un humain ou par une machine. Une activité peut être atomique ou composée. Les principaux types d'activités sont les suivants :

— **Tâche** : une tâche est représentée par un rectangle à coins arrondis tracé avec une seule ligne mince. Elle correspond à une activité atomique ayant un début et une fin. Une tâche ne peut débuter que si la tâche précédente est terminée. Il existe plusieurs types de tâches :
  — **Tâche utilisateur** : elle précise que la tâche est effectuée par un exécutant humain avec l'aide d'une application informatique.
  — **Tâche manuelle** : elle précise que la tâche doit être effectuée sans l'aide d'un moteur d'exécution de processus métier ni d'une application. Elle est donc réalisée exclusivement par un acteur humain.
  — **Tâche service** : elle utilise un service, qui peut être un service Web ou une application automatisée. Il s'agit d'une tâche automatisée sans intervention humaine.

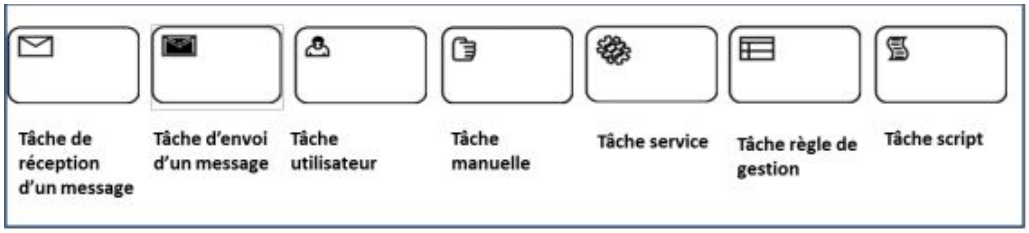


FIGURE 5 – *Les types des tâches dans BPMN 2.0.*

2. Les événements. Les événements sont représentés par des cercles. Ils correspondent à une action qui survient durant le processus. Ils ont généralement une cause et une conséquence. Il existe trois catégories d'événements : les événements de début, les événements intermédiaires et les événements de fin.



FIGURE 6 – *Événements dans BPMN 2.0.*

— **Les événements de début** : ils sont représentés par un cercle avec une seule ligne mince. Ils déclenchent une nouvelle instance de processus. Un processus peut avoir plus d'un événement de début, mais un sous-processus d'événement n'a qu'un seul événement de début.

— **Les événements intermédiaires** : ils sont représentés par un cercle avec une double ligne mince. Ils indiquent une action qui se produit ou peut se produire au cours du processus, entre le début et la fin. Ils peuvent être utilisés pour attraper un déclencheur d'événement et peuvent être placés dans le flux ou à la limite d'une activité.

— **Les événements de fin** : ils sont représentés par un cercle avec une ligne épaisse. Ils marquent la fin de l'instance de processus et ne possèdent pas de flux de séquence sortant.

3. Les passerelles. Les passerelles, ou branchements *(Gateways)*, sont représentées par un losange. Elles servent à représenter les conditions de routage entre les flux d'entrée et les flux de sortie. Les signes internes indiquent le type de contrôle du comportement du branchement.



FIGURE 7 – *Les branchements de BPMN 2.0.*

Les différents branchements possibles sont :

— **Exclusive (XOR)** : elle est utilisée pour créer des chemins alternatifs dans un flux de processus. Lors de la scission, elle dirige le flux vers une branche sortante. Lors de la fusion, elle attend qu'une branche entrante se termine avant de déclencher le flux de sortie.

— **Inclusive (OR)** : elle peut être utilisée pour créer des chemins alternatifs parallèles. Contrairement à la passerelle exclusive, toutes les expressions de condition sont évaluées. Lors de la scission, elle active une ou plusieurs branches. Lors de la fusion, elle attend que toutes les branches entrantes se terminent avant de fusionner.

— **Parallèle (AND)** : elle est utilisée pour synchroniser et créer des flux parallèles. Lors de la scission, elle active toutes les branches sortantes simultanément. Lors de la fusion, elle attend que toutes les branches entrantes se terminent.

B. Les objets de connexion

Les objets de connexion sont les éléments graphiques qui permettent de relier les objets de flux les uns aux autres pour représenter le cheminement du processus. Il existe trois types d'objets de connexion.

— **Les flux de séquence** : ils sont représentés par une flèche pleine et une ligne solide. Ils indiquent l'ordre d'exécution des actions, des événements ou des activités.



FIGURE 8 – *Les flux de séquence de BPMN 2.0.*

— **Les flux normaux** : le flux normal fait référence aux chemins du flux de séquence qui ne commencent pas à partir d'un événement intermédiaire attaché à la limite d'une activité.

C. Les partitions

Les partitions sont les éléments graphiques permettant de structurer et de regrouper les différents éléments qui composent le diagramme BPMN. Les deux types d'objets sont les participants et les couloirs.

— **Les participants (Pool)** : un pool est la représentation graphique d'un participant à une collaboration. Un participant peut être une entité partenaire spécifique, par exemple une entreprise, ou un rôle partenaire plus général, comme un acheteur, un vendeur ou un fabricant. Un pool peut avoir des détails internes sous forme d'un processus exécuté ou ne présenter aucun détail interne ; dans ce cas, il est considéré comme une « boîte noire ».

— **Les couloirs (Lane)** : un couloir est une partition verticale ou horizontale d'un pool. Il sert à organiser les activités selon un rôle, une fonction ou une responsabilité. À l'intérieur d'un même pool, y compris entre deux couloirs, l'enchaînement des activités est représenté par des flux de séquence. Les flux de message sont réservés aux échanges entre participants, donc entre pools distincts [3].



FIGURE 9 – *La représentation d'un pool avec deux lanes en BPMN 2.0.*

D. Les données

Les données sont utilisées pour expliquer quelles informations sont nécessaires dans le diagramme.



FIGURE 10 – *Les données de BPMN 2.0.*

— **Données (Data)** : objets utilisés pour représenter les données nécessaires à l'exécution d'une activité, ainsi que celles produites. Les objets de données peuvent représenter un objet singulier ou une collection d'objets.

— **Entrée de données (Data Input)** : elle correspond à une entrée externe pour l'ensemble du processus, pouvant être lue par une activité.

— **Sortie de données (Data Output)** : elle correspond au résultat de l'ensemble du processus.

E. Les artéfacts

Les artéfacts sont des objets supplémentaires qui apportent des détails complémentaires afin d'assurer une meilleure compréhension du processus. Il existe deux types d'artéfacts : le groupe et l'annotation textuelle.



FIGURE 11 – *Les artéfacts.*

— **Groupe** : un groupe est un rectangle à coins arrondis tracé avec une ligne pointillée continue. Il permet de regrouper des tâches d'une même catégorie afin de mieux les repérer visuellement sur le diagramme.
— **Annotation textuelle** : elle permet d'ajouter des commentaires pour faciliter la lecture du diagramme BPMN.

I.3.1.4. Types de diagrammes BPMN

BPMN distingue principalement les diagrammes de processus, les diagrammes de collaboration et les diagrammes de chorégraphie [3].

— Le **diagramme de processus** décrit le déroulement interne d'un processus dans un participant donné. Il est adapté lorsque l'on souhaite comprendre l'enchaînement des activités d'un service ou d'un système.



FIGURE 12 – *Exemple de diagramme de processus*

— Le **diagramme de collaboration** met en relation plusieurs participants et montre les messages qu'ils échangent. Chaque pool représente un participant. Les flux de message relient deux pools distincts ou des objets contenus dans des pools différents ; ils ne relient pas deux activités appartenant au même pool [3].

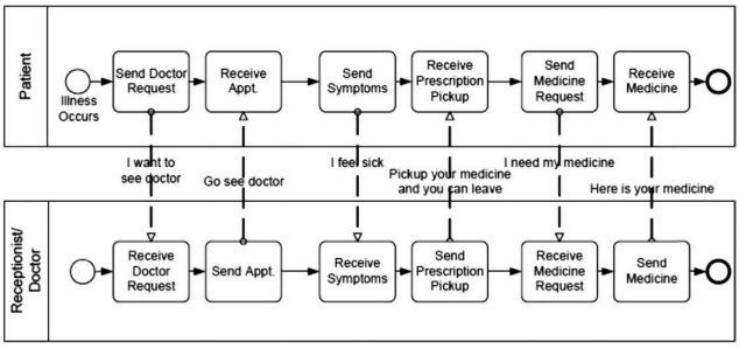


FIGURE 13 – *Exemple de diagramme de collaboration*

— Le **diagramme de chorégraphie** se concentre davantage sur les interactions entre participants que sur leur organisation interne. Il ne décrit pas l'orchestration propre à chaque participant, mais les échanges de messages qui structurent leur collaboration.

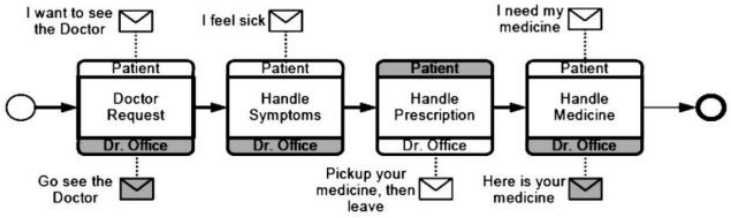


FIGURE 14 – *Exemple de diagramme de chorégraphie*

I.4. Ingénierie dirigée par les modèles

I.4.1. Principe général de l'ingénierie dirigée par les modèles

L'ingénierie dirigée par les modèles (IDM) désigne une démarche de développement dans laquelle le modèle devient un artefact de travail central. Le modèle n'est donc pas seulement un dessin destiné à expliquer une idée : il sert à décrire un système à un niveau d'abstraction choisi, à vérifier certains choix, à organiser les connaissances du domaine et, lorsque l'outillage le permet, à produire d'autres modèles ou des artefacts techniques. Cette approche répond principalement à deux difficultés du génie logiciel : la croissance de la complexité des systèmes et l'évolution rapide des technologies. En plaçant les modèles au centre du développement, l'IDM cherche à mieux séparer les préoccupations, à réduire la dépendance immédiate aux détails techniques et à rendre certaines tâches répétitives plus maîtrisables [5, 16]. L'objectif n'est pas de supprimer l'intervention humaine, mais de déplacer une partie de l'effort vers des modèles explicites, contrôlables et transformables.

I.4.2. Notions de base en ingénierie dirigée par les modèles

Un système est l'entité étudiée. Il peut être composé d'éléments reliés entre eux, posséder une organisation interne et interagir avec son environnement. Dans l'IDM, le système n'est généralement pas manipulé directement dans toute sa complexité ; il est représenté par un ou plusieurs modèles.

Un modèle est une abstraction d'un système, construite dans un objectif donné. Il retient les informations utiles pour répondre à certaines questions, tout en laissant de côté les détails qui ne sont pas nécessaires au point de vue choisi. Cette idée est importante : un modèle n'a pas vocation à tout contenir. Sa valeur dépend de sa pertinence par rapport au besoin d'analyse, de conception ou de vérification. Jézéquel, Combemale et Vojtisek définissent ainsi le modèle comme un ensemble de faits caractérisant un aspect d'un système selon un objectif précis [5].

Un méta-modèle définit le langage dans lequel les modèles sont exprimés. Il précise les concepts autorisés, leurs relations et les contraintes qui structurent les modèles valides. Le méta-modèle joue donc un rôle de cadre : il rend les modèles manipulables par des outils, car il décrit leur structure attendue. Au-dessus, le méta-méta-modèle fournit le langage permettant de définir les méta-modèles. L'Object Management Group (OMG) organise cette logique selon quatre niveaux : M0 pour le système réel, M1 pour les modèles, M2 pour les méta-modèles et M3 pour le méta-méta-modèle, généralement associé à MOF, Meta-Object Facility [5, 17].

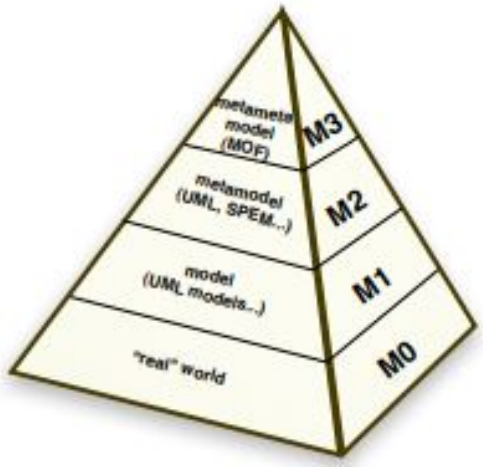


FIGURE 15 – *Pyramide de modélisation de l'OMG, d'après [5].*

I.4.3. Transformation de modèles

La transformation de modèle consiste à produire un modèle cible à partir d'un modèle source, selon des règles définies. Ces règles décrivent les correspondances

entre les éléments du modèle d'origine et ceux du modèle obtenu. Une transformation peut être endogène lorsque les modèles source et cible sont conformes au même méta-modèle ; elle est exogène lorsqu'ils relèvent de méta-modèles différents. Elle peut aussi être horizontale lorsqu'elle reste au même niveau d'abstraction, ou verticale lorsqu'elle fait évoluer le niveau d'abstraction, par raffinement ou par abstraction [5].

Les transformations occupent une place centrale dans l'IDM parce qu'elles donnent un caractère productif aux modèles. Elles peuvent servir à raffiner une spécification, restructurer une représentation, migrer d'une technologie à une autre, générer du texte ou produire du code. Les deux grandes familles généralement retenues sont les transformations modèle-vers-modèle (M2M) et modèle-vers-texte (M2T). Les standards et langages de transformation, comme QVT, ATL ou les approches par templates, cherchent à formaliser ces passages afin qu'ils soient reproductibles et moins dépendants d'une manipulation manuelle [5, 18].

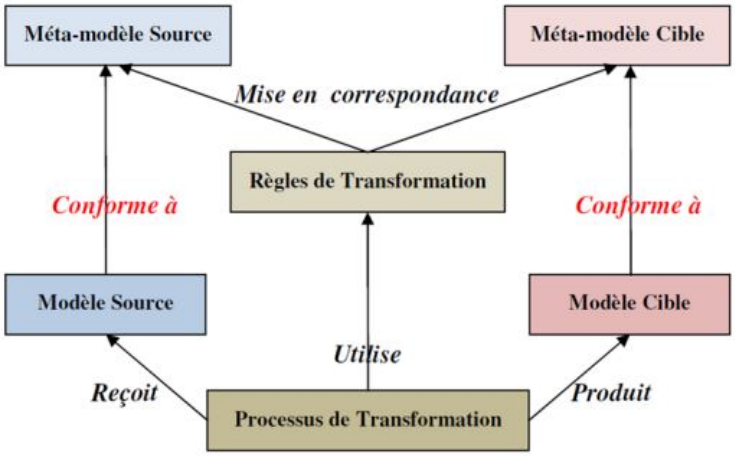


FIGURE 16 – *Schéma de base d'une transformation de modèle [16].*

I.4.4. Model-Driven Architecture

La Model-Driven Architecture (MDA) est une initiative de l'OMG qui applique les principes de l'IDM à l'architecture logicielle. Elle vise à décrire une application en distinguant les modèles indépendants de la plateforme et les modèles liés à une plateforme particulière.

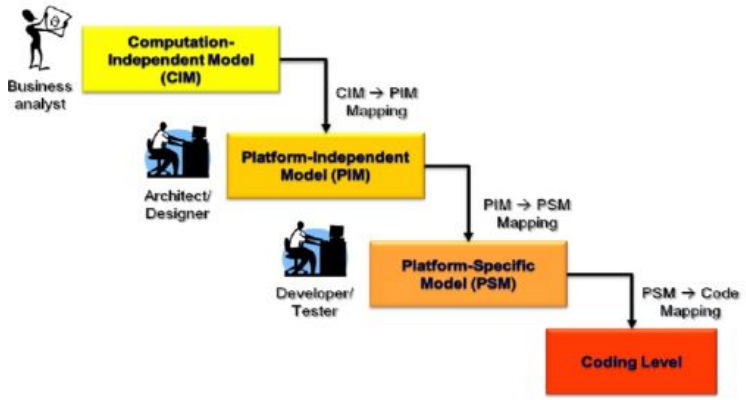


FIGURE 17 – *Illustration des trois niveaux d'abstraction de la MDA [16].*

— **CIM** ***(Computation Independent Model)*** : il décrit les besoins métier et les objectifs sans se préoccuper des aspects techniques. C'est le niveau où l'on formalise ce que le système doit faire, indépendamment de la façon dont il sera implémenté.
— **PIM** ***(Platform Independent Model)*** : ce modèle représente l'architecture et les fonctionnalités du système sans faire référence à une technologie ou une plateforme donnée. Il constitue un pont entre les exigences métier et la solution technique.
— **PSM** ***(Platform Specific Model)*** : il traduit le PIM en une implémentation concrète, en intégrant les contraintes et les choix technologiques spécifiques comme le framework, le langage ou l'environnement d'exécution.

L'intérêt de la MDA est de préserver les décisions de conception à un niveau moins fragile que le code. Lorsque les technologies changent, les modèles indépendants peuvent être conservés plus longtemps, tandis que les transformations et les modèles spécifiques sont adaptés aux nouvelles plateformes. La MDA ne se limite donc pas à une génération automatique de code ; elle propose une organisation du développement où la séparation entre abstraction, plateforme et implémentation devient explicite.

I.5. Applications web métier

I.5.1. Généralités sur le développement web

I.5.1.1. Historique du développement web

Le développement web est né avec un objectif essentiellement documentaire : publier des informations et permettre leur consultation à distance. Dans cette pre-

mière logique, le navigateur demandait une page, le serveur retournait un document HTML, puis le navigateur l'affichait. Les fondements du Web restent liés à cette organisation : identification des ressources, liens hypertextes, représentations et échanges entre clients et serveurs [19].

L'évolution des usages a progressivement transformé le Web en plateforme applicative. Les pages statiques ont été complétées par des formulaires, des espaces authentifiés, des tableaux de bord, des services de paiement, des plateformes universitaires et des applications administratives. Le navigateur n'est alors plus seulement un lecteur de documents ; il devient l'interface d'un système logiciel distant. Cette évolution a donné plus d'importance aux langages côté client, aux serveurs d'application, aux bases de données, aux API, à la sécurité et à la maintenabilité.

HTTP demeure le support principal de ces échanges. Le RFC 9110 le définit comme un protocole applicatif fondé sur des requêtes, des réponses et des méthodes comme `GET`, `POST`, `PUT`, `PATCH` ou `DELETE` [20]. Cette base explique pourquoi les applications web modernes sont généralement construites autour de routes, de ressources, d'opérations métier et de formats de représentation comme HTML ou JSON.

I.5.1.2. Définition d'application web et concepts du web

Définition

Une application web peut être définie comme un logiciel accessible à partir d'un navigateur ou d'un client reposant sur les technologies du Web. Contrairement à un site essentiellement informatif, elle prend en charge des interactions avec l'utilisateur, traite des données, applique des règles métier et produit des résultats exploitables. Ainsi, une plateforme de demande d'attestation ne se limite pas à afficher des pages : elle permet de soumettre une demande, de suivre son état, de contrôler les droits d'accès et de générer un document. Elle relève donc pleinement d'une application web métier.

Concepts du web

La conception d'une application web repose sur des choix architecturaux qui influencent directement sa maintenabilité, son évolutivité et la séparation des responsabilités. Le modèle client-serveur constitue la base de ce fonctionnement : le client, généralement un navigateur, transmet les actions de l'utilisateur, tandis que le serveur traite les requêtes, accède aux données et retourne les réponses appropriées [21]. Cette séparation est particulièrement importante dans les applications

métier, où plusieurs utilisateurs interagissent avec les mêmes ressources selon des droits et des règles différents.

Pour améliorer l'organisation interne du système, les applications web modernes s'appuient souvent sur une architecture en couches. La couche de présentation gère les interfaces et les formulaires, la couche métier applique les règles du domaine, et la couche d'accès aux données assure la communication avec la base. Cette structuration limite le couplage entre les composants et facilite les évolutions du système. Le patron Modèle-Vue-Contrôleur s'inscrit dans cette logique en distinguant les données et règles du domaine, leur présentation à l'utilisateur et le traitement des actions déclenchées par celui-ci [22].

Dans les architectures récentes, les applications monopages *(Single Page Applications)* renforcent la séparation entre le client et le serveur en confiant une partie importante de l'interface au navigateur. Elles offrent une expérience plus fluide, mais exigent une maîtrise accrue de la complexité du frontend, notamment lorsque l'application doit représenter des tâches, des états de dossier ou des actions liées à un processus métier [23]. Les API web jouent alors un rôle central en exposant les fonctionnalités du système sous forme de ressources accessibles, comme `/demandes`, `/documents` ou `/taches`. Le style REST, formalisé par Fielding, a fortement contribué à cette organisation des échanges autour des ressources, des représentations et d'interactions sans état [24].

Ainsi, le développement d'une application web ne consiste pas seulement à produire des interfaces. Il implique de définir une architecture capable d'articuler présentation, logique métier, persistance des données et échanges entre composants. Cette base conceptuelle est essentielle pour comprendre les applications web orientées processus, dont le comportement dépend étroitement de l'évolution du workflow.

I.5.1.3. Frameworks d'application web

Un framework d'application web fournit un cadre de développement, des composants réutilisables et des conventions pour construire une application. Il évite de réécrire à chaque projet les mêmes mécanismes : routage, contrôleurs, accès aux données, gestion des sessions, validation, rendu des vues, authentification ou traitement des erreurs. Son intérêt n'est pas seulement de gagner du temps ; il permet aussi d'organiser le code de manière plus régulière et plus maintenable.

Frameworks de développement web back-end

Les frameworks back-end structurent la partie serveur de l'application. Ils organisent les routes, les contrôleurs, les services, les modèles de données et les mé-

canismes de sécurité. Dans une application métier, c'est dans cette partie que sont vérifiées les règles comme l'autorisation d'un utilisateur, la validité d'une demande ou le passage d'un dossier d'un état à un autre. Ils peuvent aussi faciliter l'accès aux bases de données, la journalisation, la gestion des erreurs et l'exposition d'API. Des environnements comme Spring, Django, Laravel ou Express illustrent cette famille d'outils.

Frameworks de développement web front-end

Les frameworks front-end structurent l'interface exécutée dans le navigateur. Ils aident à construire des composants réutilisables, gérer les formulaires, afficher les données et réagir aux actions de l'utilisateur. Dans une application orientée processus, cette partie est importante parce qu'elle rend visibles les tâches à traiter, les états du dossier, les messages d'erreur et les actions autorisées. Leur rôle est donc aussi ergonomique : ils aident à présenter le workflow de façon compréhensible pour chaque acteur. Des outils comme Angular, React ou Vue.js sont représentatifs de cette logique côté client.

I.5.1.4. Sécurité dans les applications web

Les politiques de contrôle d'accès permettent de définir les règles qui encadrent l'accès aux informations au sein d'une entreprise. Dans les applications web, elles contribuent à sécuriser la manipulation des données sensibles, notamment les données des utilisateurs. Parmi ces politiques, le modèle RBAC, ou contrôle d'accès basé sur les rôles, est largement utilisé, car il permet d'organiser les droits d'accès en fonction des responsabilités attribuées aux utilisateurs.

Le principe du RBAC consiste à associer des permissions à des rôles, puis à affecter les utilisateurs aux rôles correspondant à leurs besoins. Cette approche simplifie la gestion de la sécurité, car les administrateurs peuvent créer des rôles, leur attribuer des autorisations et contrôler ainsi l'accès aux ressources. Elle favorise également des principes essentiels comme le moindre privilège, la séparation des tâches et la protection des informations sensibles.

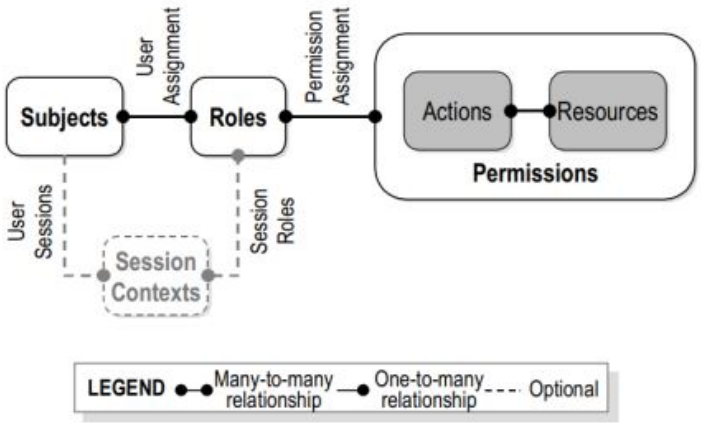


FIGURE 18 – *Cœur du modèle RBAC, adapté de [25]*

I.5.2. Applications web métier

I.5.2.1. Définitions et principe

Une application web orientée processus, ou *process-aware web application*, est une application dont le comportement est organisé autour d'un processus métier explicite. Les écrans, les actions disponibles, les rôles des utilisateurs et les changements d'état suivent l'avancement d'un workflow. Cette idée rejoint la définition des systèmes d'information orientés processus, dans lesquels les activités, les acteurs et les règles d'enchaînement occupent une place centrale [1, 2].

Dans une application classique, le développeur peut coder directement les conditions de passage d'un état à un autre. Dans une application orientée processus, ces passages sont plutôt décrits par un modèle, souvent en BPMN. Le modèle précise les tâches à réaliser, les décisions possibles, les événements attendus et les responsabilités des acteurs. L'application web fournit ensuite les interfaces, les services et les données nécessaires à l'exécution concrète de ce processus.

I.5.2.2. Moteur de processus

Un moteur de processus est un composant logiciel chargé d'exécuter un modèle de processus. Lorsqu'un processus BPMN exécutable est déployé, il peut créer des instances, enregistrer leurs variables, distribuer des tâches humaines, déclencher des services automatiques et suivre l'historique d'exécution [26]. Il joue donc un rôle d'orchestration : il indique ce qui doit être fait, par qui, à quel moment et selon quelles conditions.

Dans une application web orientée processus, le moteur ne remplace pas le frontend, le backend ni la base de données. Il coordonne le déroulement du travail. Le frontend permet aux utilisateurs d'interagir avec les tâches, le backend applique

les règles propres à l'application et dialogue avec le moteur, tandis que la base de données conserve les informations métier. Cette répartition évite de confondre l'exécution du workflow avec la construction complète de l'application.

I.6. Conclusion

Dans ce chapitre, nous avons présenté les concepts nécessaires à la suite du mémoire : processus métier, BPM, BPMN, développement web et applications web métier. Nous avons montré qu'un modèle BPMN peut guider l'organisation du travail, mais qu'une application web doit aussi gérer l'interface, les données, la sécurité et l'exécution technique. Le chapitre suivant s'appuie sur ces bases pour analyser les travaux existants.

II

CHAPITRE

ÉTAT DE L'ART SUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR DES MODÈLES

SOMMAIRE

II.1 - Introduction 26
II.2 - Démarche d'analyse des travaux existants 26
II.3 - Revue de la littérature 27
II.4 - Discussion et positionnement de la recherche 37
II.5 - Conclusion 40

II.1. Introduction

La génération d'applications web à partir de modèles de processus se situe au croisement du BPM, de BPMN et du développement web. BPMN permet de représenter le déroulement du travail, mais il ne suffit pas à décrire les interfaces, les données, les règles d'accès et l'architecture d'une application complète [3].

Ce chapitre analyse donc les travaux qui rapprochent les modèles de processus des artefacts logiciels, afin d'en dégager les apports, les limites et le positionnement de notre recherche.

II.2. Démarche d'analyse des travaux existants

La revue n'a pas cherché à rassembler tous les travaux sur BPMN, mais à analyser ceux qui répondent à une question centrale : comment un modèle de processus

peut-il être transformé en application web ou en artefacts logiciels utilisables ? Le corpus retenu privilégie donc les travaux portant sur les moteurs de workflow, les transformations BPMN vers d'autres modèles, la génération de code, les applications web générées, les plateformes orientées processus et les approches low-code. La démarche s'inspire des recommandations de Kitchenham et Charters ainsi que de l'esprit de PRISMA, sans constituer une revue systématique exhaustive [27, 28].

L'analyse des travaux s'est faite en trois étapes : comprendre l'objectif de chaque approche, identifier les artefacts réellement produits et examiner les limites récurrentes. Ces limites concernent notamment l'insuffisance d'informations dans les modèles BPMN, la couverture partielle des applications, la dépendance à certaines plateformes, les difficultés de régénération et la cohabitation entre code généré et code manuel.

TABLE II – *Grille concrète de lecture des travaux étudiés*

Point observé	Question posée pendant la lecture
Point de départ	Le travail part-il d'un modèle BPMN seul, d'un processus enrichi, d'un modèle de données ou d'un autre artefact ?
Résultat obtenu	Produit-il une interface, une API, une application exécutable, un prototype ou seulement une méthode de transformation ?
Place de la plateforme	Le résultat dépend-il fortement d'un moteur, d'un framework ou d'un environnement low-code ?
Preuve apportée	Les auteurs valident-ils leur proposition par un cas d'étude, une expérimentation, une comparaison ou une revue ?
Limite retenue	Que reste-t-il à faire pour obtenir une application complète, maintenable et adaptable ?

La revue qui suit présente donc les travaux par familles, puis met en évidence les limites qui se dégagent de leur comparaison.

II.3. Revue de la littérature

II.3.1. Systèmes d'information orientés processus et plateformes d'exécution

Les systèmes d'information orientés processus placent le déroulement du travail au centre de l'organisation du logiciel. Ils s'inscrivent dans le domaine plus large de la gestion des processus métier, qui couvre la découverte, la modélisation, l'exécution, le suivi et l'amélioration des processus [1, 29]. Dans cette perspective,

le modèle ne sert plus uniquement à documenter une organisation : il peut devenir une définition exécutable utilisée par un moteur de processus.

II.3.1.1. Moteurs BPMS et plateformes d'exécution

Les systèmes de gestion des processus métier, ou *Business Process Management Systems* (BPMS), tels que Camunda, Flowable, jBPM ou Activiti interprètent des définitions de processus, créent des instances, distribuent des tâches et conservent un historique d'exécution [26, 30–32]. Leur principal apport réside dans la séparation entre la logique d'orchestration et les services appelés au cours du processus. Cette séparation rend le déroulement visible et facilite son suivi. Elle impose néanmoins aux applications clientes de manipuler les notions propres au moteur, notamment les instances, les variables et les tâches. La plateforme exécute ainsi le processus sans nécessairement produire l'ensemble de l'application qui l'entoure.

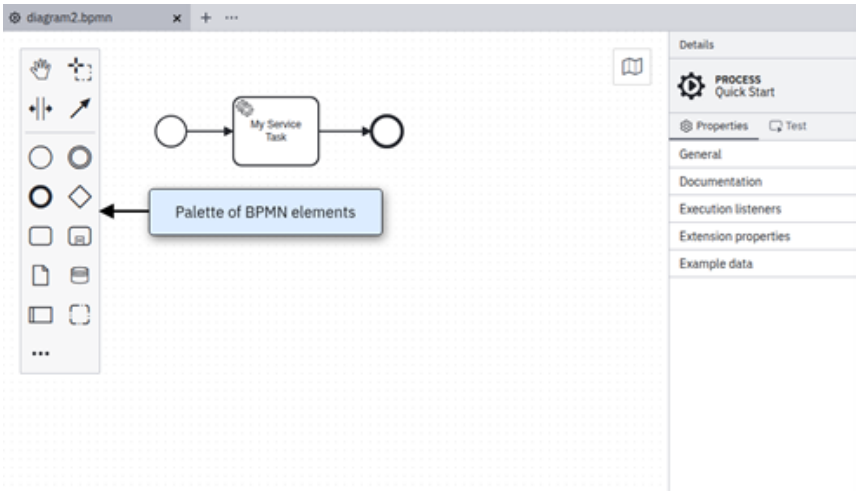


FIGURE 19 – *Modélisation d'un processus BPMN exécutable dans Camunda Modeler, d'après la documentation Camunda [26].*

La figure 19 montre le point d'entrée d'un moteur comme Camunda : un processus BPMN enrichi de propriétés exécutables. Une fois déployé, le moteur crée les instances, affecte les tâches et suit l'exécution, mais il ne génère pas à lui seul toute l'application web qui l'exploite.

II.3.1.2. Architecture web pilotée par BPMN

Schäffer et al. proposent une architecture de référence et une méthode de développement agile pour une plateforme web pilotée par BPMN [33]. Leur travail montre comment associer une application web, un moteur de processus et des services afin de conserver le processus comme fil conducteur de l'implémentation.

L'approche apporte une organisation cohérente aux systèmes dont le comportement dépend fortement d'un workflow. Elle reste toutefois centrée sur l'exécution et l'intégration du processus : les autres dimensions de l'application doivent toujours être conçues et développées autour de cette architecture.

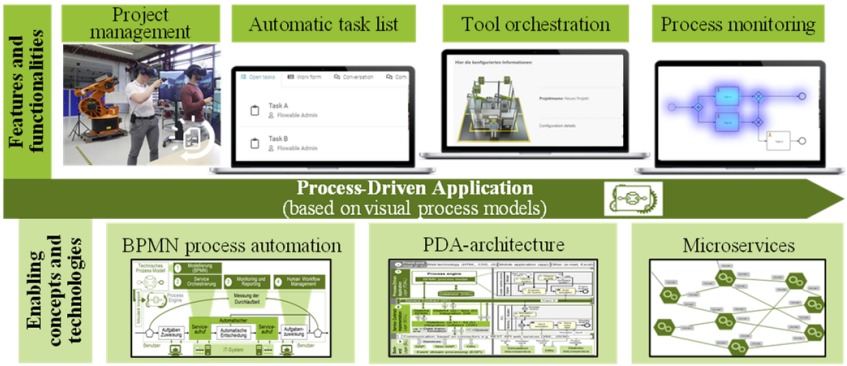


FIGURE 20 – *Fonctionnalités et technologies d'une application web pilotée par processus [33].*

La figure 20 montre qu'une application pilotée par processus combine moteur BPMN, tâches, orchestration, supervision et services. Le moteur de workflow y apparaît central, mais il doit être entouré de composants applicatifs et d'intégration.

II.3.1.3. WebWorkFlow

WebWorkFlow adopte une direction différente. Hemel et al. considèrent qu'un langage de workflow isolé n'est pas suffisant pour générer une application web complète. Ils étendent donc WebDSL avec des abstractions de workflow, puis traduisent ces abstractions vers les constructions du langage sous-jacent [34]. Cette intégration permet de générer des applications personnalisées tout en couvrant les principaux patrons de contrôle de workflow.

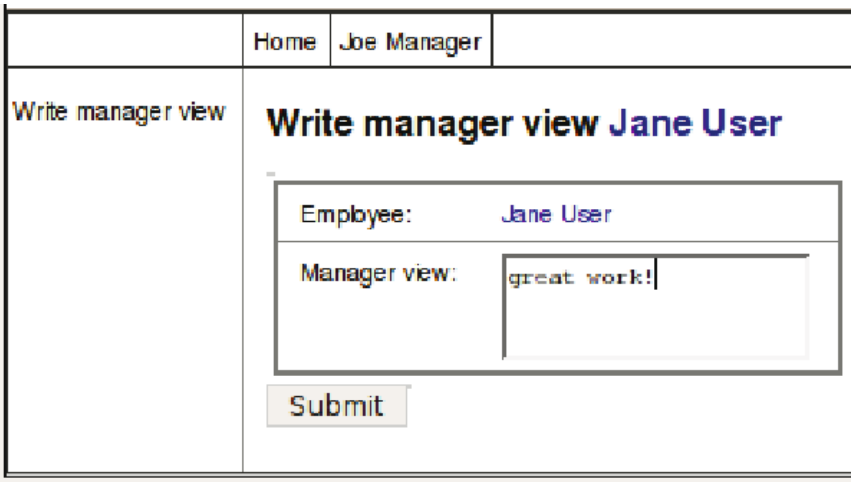


FIGURE 21 – *Interface générée dans WebWorkFlow pour une tâche de workflow, extraite de [34].*

La figure 21 illustre le résultat attendu : une tâche de workflow devient une interface exploitable. L'apport est concret, mais la portée de la génération reste liée au langage et aux abstractions de WebDSL.

II.3.1.4. Plateforme AKIP

La plateforme ouverte AKIP, nom donné par ses auteurs, cherche à produire des applications web orientées processus à partir de BPMN exécutable et de spécifications complémentaires [35]. Son architecture associe un générateur, un frontend, un backend, un moteur de processus, des connecteurs et plusieurs fonctions transversales. Les auteurs répondent notamment aux limites observées dans des plateformes existantes : dépendance envers un fournisseur, interfaces peu flexibles, difficultés d'intégration et technologies devenues anciennes.

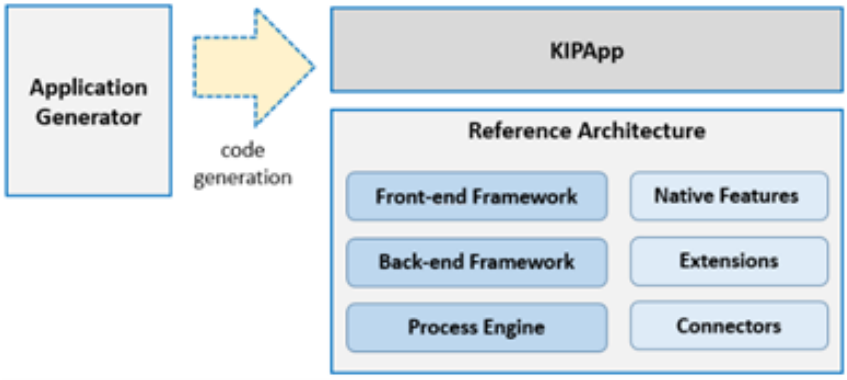


FIGURE 22 – *Architecture et principe de génération d'une application dans AKIP, adaptés de [35].*

La figure 22 montre une génération organisée autour d'une architecture de référence. AKIP produit une base applicative moderne, mais son évaluation reste limitée à une organisation et certaines constructions BPMN ne sont pas encore couvertes.

Ces travaux montrent que l'exécution d'un processus est aujourd'hui bien maîtrisée. Ils montrent également que le moteur ne résout qu'une partie du problème applicatif. Plus une proposition vise une application complète, plus elle doit associer au workflow des informations sur le domaine, les interfaces et l'intégration technique.

II.3.2. Dérivation d'artefacts applicatifs à partir de BPMN

Une deuxième famille de recherches utilise BPMN comme point de départ pour dériver des artefacts web précis. Ces travaux ne cherchent pas toujours à générer l'ensemble d'un système. Ils étudient plutôt la manière dont les éléments du pro-

cessus peuvent fournir des informations utiles à la conception d'interfaces, d'API ou de prototypes.

II.3.2.1. Transformation vers des modèles d'interaction

Sajji et al. proposent une transformation de BPMN vers l'*Interaction Flow Modeling Language* (IFML) dans le cadre de la *Model Driven Architecture* (MDA) [7]. BPMN représente le niveau indépendant du calcul, tandis qu'IFML décrit les vues, les événements utilisateur et les flux d'interaction au niveau indépendant de la plateforme. Le travail établit des règles de correspondance entre les éléments du processus et ceux de l'interface. Il confirme que la structure du workflow peut guider la construction de la navigation, mais que BPMN doit être complété pour exprimer les détails de l'interaction.

Ces mêmes auteurs poursuivent cette logique en étudiant la transformation d'IFML vers Symfony [36]. L'intérêt de ce prolongement est de montrer que le modèle d'interaction peut servir de point de départ à une génération orientée framework web concret. La transformation s'appuie sur des règles ATL, c'est-à-dire un langage utilisé pour définir des transformations entre modèles, pour passer d'un modèle IFML indépendant de la plateforme vers un modèle Symfony plus proche de l'implémentation.

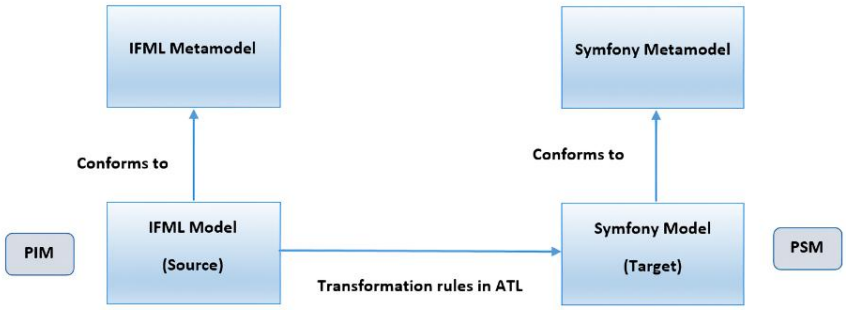


FIGURE 23 – *Transformation d'un modèle IFML vers un modèle Symfony, adaptée de [36].*

La figure 23 illustre la logique MDA : IFML sert de modèle indépendant, puis Symfony rapproche la représentation de l'implémentation. Cette continuité réduit l'écart avec le code, mais renforce aussi la dépendance aux conventions du framework cible.

Laaz et al. enrichissent les modèles BPMN avec des ontologies de domaine avant de produire des modèles IFML [37]. Les ontologies permettent de préciser les concepts métier que le processus manipule sans les surcharger directement dans le diagramme. Cette proposition répond à une faiblesse récurrente de BPMN : les activités et les flux sont explicites, mais la sémantique des données demeure souvent implicite. La qualité du résultat dépend toutefois de la disponibilité et de la

qualité de l'ontologie, dont la construction représente elle-même un effort important.

II.3.2.2. Génération de prototypes et d'API

Díaz et al. associent les processus BPMN à des modèles structurels afin de générer des prototypes d'interfaces graphiques [6]. Leur travail montre l'intérêt d'un prototype obtenu tôt dans le cycle de développement : il permet de confronter le processus modélisé aux interactions attendues par les utilisateurs. Le terme de prototype marque cependant la limite assumée de la proposition. L'interface obtenue facilite la validation, mais elle ne constitue pas encore une application complète prête à être exploitée.

Pietruck et Steffens s'intéressent à la génération d'API conformes au style architectural *Representational State Transfer* (REST) pour les moteurs de workflow, à partir de modèles de processus et de données [38]. Les API produites exposent des opérations plus proches du domaine métier que les interfaces génériques d'un moteur. Cette abstraction réduit le couplage entre les applications clientes et les concepts techniques de son environnement d'exécution. Elle couvre cependant la couche d'accès au processus et aux données, sans générer les interfaces utilisateur ni l'ensemble des fonctions transversales.

II.3.2.3. Vers une génération intégrée des artefacts applicatifs

Hoyos et Restrepo-Calle proposent de générer automatiquement un prototype d'application web orientée processus à partir d'une spécification rédigée dans un langage naturel restreint [39]. Leur chaîne analyse la spécification, extrait les informations utiles, puis produit des classes du domaine, des formulaires, des vues et des contrôleurs.

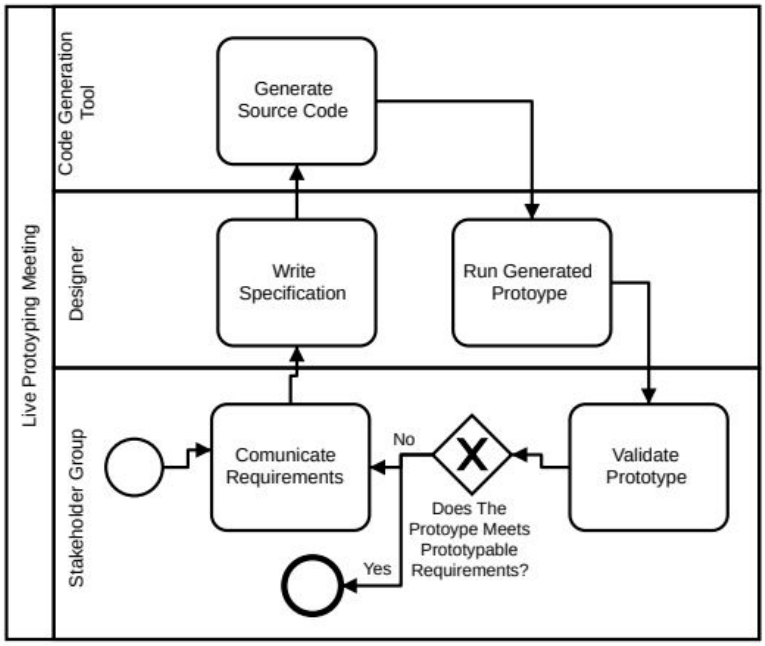


FIGURE 24 – *Cycle de prototypage rapide d'applications web proposé par [39].*

```
@app.route('/projectList/<pid>/showSelectedProject',
methods=['GET','POST'])
def projectList_showSelectedProject(pid):
    viewparams={}
    data = get_instance_data(pid)
    formdata = request.form.copy()
    for file in request.files:
        formdata[file] = request.files[file].filename
    request.form = formdata
    project = session.query(Project).filter(
       Project.id==data.get("selected_Project",0)
    ).first()
    viewparams['project']=project
    task_list = session.query(Task).filter(
        Task.project_id==data.get('selected_Project',0)
    ).all()
    viewparams['task_list']=task_list
```

```
<div class="panel-heading">Show Selected Project</div>
<div class="panel-body">
{% for field in project.__table__.columns._data.keys()%}
<div class="form-group">
        <b>{{field}}</b><br /> {{project[field]}}
</div>
{% endfor %}
<div class="col-lg-12">
<div class="table-responsive">
<table class="table">
<tr>
{% for prop in task_list[0].__mapper__.columns.keys() %}
<th>{{prop}}</th>
{% endfor %}
</tr>
{% for ent in task_list %}
<tr>
{% for prop in ent.__mapper__.columns.keys() %}
<td>{{ent[prop]}}</td> {% endfor %}
</tr>
{% endfor %}
</table></div></div></div>
```

FIGURE 25 – *Extraits de code généré pour le contrôleur et la vue dans l'approche de [39].*

La figure 25 complète le cycle précédent en montrant deux artefacts produits : une route côté contrôleur et un gabarit de vue. Elle confirme que l'approche génère un prototype exploitable, même si la qualité finale dépend encore des règles de génération.

Uyanik et Sayar développent un environnement web qui réunit conception interactive du processus, gestion des flux et génération automatique de code [40]. Dans leur contexte d'évaluation, les auteurs rapportent une amélioration de 98,68 % du

processus de développement, de 95,84 % pour les conditions de test et de 36,01 % concernant la taille du code. Ces résultats soulignent le potentiel d'une chaîne intégrée pour des applications principalement organisées autour de formulaires et de processus. Ils restent toutefois liés au protocole et au domaine évalués ; ils ne permettent pas d'établir que les mêmes gains seraient obtenus pour des architectures plus variées.

Pris ensemble, ces travaux confirment que BPMN contient des informations exploitables pour produire plusieurs parties d'une application web. Ils montrent aussi qu'une transformation ciblée conserve généralement une couverture ciblée : l'interface, l'API ou le prototype progresse, tandis que d'autres préoccupations restent hors du périmètre.

II.3.3. Ingénierie dirigée par les modèles et génération de systèmes orientés processus

L'ingénierie dirigée par les modèles considère les modèles comme des artefacts productifs pouvant être validés, transformés et reliés au code. Cette orientation est particulièrement adaptée aux systèmes orientés processus, car elle permet d'associer plusieurs descriptions spécialisées au lieu d'exiger qu'un seul modèle représente toute l'application.

II.3.3.1. Composition de langages et génération progressive

Drave et al. proposent une démarche générative pour les systèmes d'information orientés processus [9]. Leur approche combine un langage textuel pour BPMN avec des langages spécifiques décrivant notamment les structures de données et les rôles. Les modèles sont contrôlés, transformés en représentations intermédiaires, puis utilisés pour générer un système extensible. Cette composition de langages vise à couvrir à la fois les activités manuelles et automatisées, là où de nombreuses propositions se concentrent sur une seule de ces dimensions.

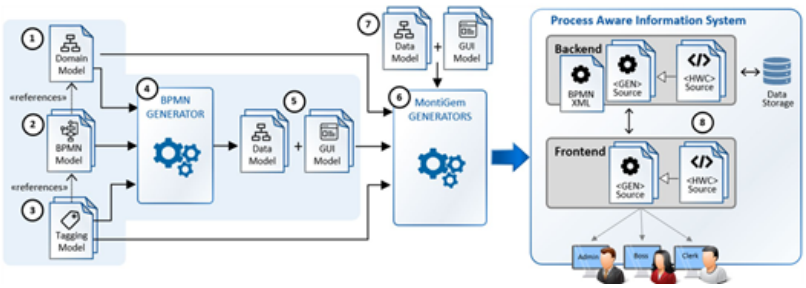


FIGURE 26 – *Génération en deux étapes d'un système d'information orienté processus, adaptée de [9].*

La figure 26 montre une génération en deux étapes qui facilite le contrôle des transformations et la réutilisation des informations. Sa limite tient à la couverture incomplète de BPMN et au coût de maintenance des langages spécifiques mobilisés.

II.3.3.2. Gabarits et compléments aux modèles BPMN

La génération fondée sur des templates, c'est-à-dire des gabarits de fichiers complétés automatiquement, reste l'une des techniques les plus utilisées pour passer d'un modèle vers du code. L'étude de cartographie menée par Syriani et al. montre que cette technique est répandue et qu'elle facilite l'adaptation aux technologies cibles [41]. Dans le cadre d'une génération d'applications web métier, les gabarits ne sont toutefois utiles que s'ils traduisent correctement les informations du processus en artefacts concrets : routes, contrôleurs, formulaires, services, règles métier et accès aux données. Leur maintenance devient alors un enjeu, car ils doivent évoluer avec les cadres logiciels, les conventions de développement et la structure des modèles d'entrée.

Plusieurs travaux montrent que le modèle de processus doit souvent être complété pour produire une application exploitable. Fedeli et al. introduisent des concepts propres aux environnements enrichis par l'Internet des objets [42], tandis que Salado-Cid et al. insistent sur la représentation explicite des données, des opérations et des dépendances dans les workflows intensifs en données [43]. Ces contributions ne visent pas directement les applications web métier classiques, mais elles soulignent une limite importante de BPMN : le flux de contrôle ne suffit pas toujours à décrire les données manipulées, les interactions utilisateur et les contraintes métier nécessaires à la génération.

Les travaux analysés font apparaître une limite récurrente : BPMN décrit efficacement le flux de contrôle, les activités, les événements, les passerelles et la répartition des responsabilités, mais il ne porte pas, à lui seul, toutes les informations nécessaires à la génération d'une application web métier. Dès que l'on cherche à produire des formulaires, à organiser la persistance, à contrôler les droits d'accès, à définir la visibilité des données ou à relier les sorties d'une activité aux entrées d'une autre, le modèle de processus doit être complété. Selon les approches, ce complément prend la forme de modèles de données, de modèles d'interaction, de métadonnées, de règles propres au domaine, de mappings ou de spécifications supplémentaires. Cette observation montre qu'une approche de génération ne peut pas demander à BPMN de tout décrire ; elle doit plutôt conserver BPMN comme mo-

dèle du processus et lui associer une description complémentaire des informations applicatives.

II.3.4. Maintenabilité du code généré et mise en perspective avec le low-code

La plupart des travaux de génération accordent une attention importante au premier passage du modèle vers le code. Or, une application continue d'évoluer après sa génération initiale. Les développeurs ajoutent des règles propres au domaine, corrigent des comportements et adaptent les interfaces. En parallèle, les processus et les technologies cibles changent. La coexistence du code généré et du code écrit manuellement devient alors une question centrale.

II.3.4.1. Gestion conjointe du code généré et du code manuel

Greifenberg et al. comparent plusieurs mécanismes d'intégration du code manuscrit et du code généré, notamment la séparation des fichiers, l'héritage, la délégation et les zones protégées [44]. Leur analyse montre qu'aucun mécanisme ne répond parfaitement à toutes les situations. Certains protègent clairement les modifications manuelles, mais imposent une architecture plus complexe. D'autres offrent davantage de souplesse, au prix d'un risque de conflit lors de la régénération. Le choix doit donc être prévu dès la conception du générateur.

Rahad et al. étudient la place du développeur dans la boucle de l'ingénierie dirigée par les modèles [45]. Ils montrent que l'automatisation ne supprime pas l'intervention humaine : celle-ci se déplace vers la vérification des modèles, l'extension du code et la résolution des écarts entre le résultat généré et le besoin réel. SeamlessMDD poursuit une préoccupation similaire en cherchant à intégrer plus naturellement le code généré aux pratiques habituelles de développement [46]. Ces contributions rappellent que la qualité d'une chaîne générative se mesure aussi après la première génération.

Les travaux consacrés à l'intégration du code généré et du code manuscrit montrent que plusieurs stratégies peuvent être utilisées : séparation des fichiers, héritage, délégation, zones protégées, points d'extension ou mécanismes d'override. Aucune de ces stratégies n'est universelle. Leur pertinence dépend du type d'artefacts générés, du niveau de personnalisation attendu et de la fréquence des régénérations. Cette difficulté confirme que la maintenabilité doit être prise en compte dès la conception d'une chaîne de génération.

II.3.4.2. Mise en perspective avec les approches low-code

Les plateformes low-code cherchent également à réduire l'effort de développement grâce à des modèles visuels, composants et connecteurs [47, 48]. Elles partagent avec l'IDM une recherche d'abstraction et d'automatisation. Toutefois, elles restent souvent dépendantes des plateformes, de leurs métamodèles, de leurs règles de gouvernance et de leurs connecteurs. Dans le cadre de ce mémoire, l'enjeu est différent : il s'agit de conserver des sources explicites, des transformations identifiables et des artefacts dont l'origine peut être inspectée. Cette différence situe le travail dans une logique d'ingénierie dirigée par les modèles plutôt que dans une démarche low-code généraliste.

II.4. Discussion et positionnement de la recherche

La littérature étudiée confirme qu'il est possible d'utiliser un modèle de processus pour exécuter un workflow, produire des interfaces, exposer des API ou générer une base applicative. Elle ne fait toutefois pas apparaître une réponse unique couvrant, avec le même niveau de maturité, toutes les dimensions d'une application web. Les propositions se distinguent surtout par le compromis qu'elles établissent entre couverture, automatisation, flexibilité et dépendance technologique.

Le tableau III montre que les travaux étudiés apportent des réponses utiles, mais rarement complètes. Certains exécutent le processus, d'autres génèrent une interface, une API ou un prototype. Les approches les plus larges produisent une base applicative, mais elles restent liées à une architecture, à un langage ou à une plateforme particulière.

TABLE III – *Récapitulatif des travaux étudiés*

Travail	**Portée et résultat produit**	**Évaluation présentée**	**Limite principale**
WebWorkFlow [34]	Intégration d'abstractions de workflow à WebDSL pour générer des applications web personnalisées	Couverture des principaux patrons de contrôle de workflow	Dépendance au langage WebDSL et à ses abstractions
Sajji et al. [7, 8, 36]	Transformation de BPMN vers IFML, puis d'IFML vers des technologies web comme WebRatio ou Symfony	Cas d'étude d'une application de service après-vente et règles de transformation vers Symfony	Couverture principalement limitée aux interactions et dépendance aux conventions de la plateforme cible
Hoyos et Restrepo-Calle [39]	Génération de classes, formulaires, vues et contrôleurs depuis un langage naturel restreint	Étude de cas démontrant la génération rapide d'un prototype	Langage d'entrée contraint et résultat orienté prototypage
Pietruck et Steffens [38]	Génération d'API REST orientées métier pour des moteurs de workflow	Démonstration de la transformation de modèles de processus et de données	Interfaces utilisateur et fonctions transversales hors du périmètre
Delgado et al. [49]	Gestion dirigée par les modèles des familles de processus BPMN	Représentation de variantes proches d'un même processus	Configuration et contrôle de cohérence plus complexes
AKIP [35]	Génération d'une application web fonctionnelle intégrant interface, serveur et moteur de processus	Étude industrielle portant sur trois processus	Personnalisations métier nécessaires et couverture BPMN partielle
Uyanik et Sayar [40]	Environnement intégré de conception de processus et de génération de code	Gains importants rapportés pour le développement, les tests et la taille du code	Résultats liés à un contexte expérimental spécifique
Travaux low-code [48, 50–53]	Usage de modèles visuels ou d'entreprise pour accélérer la construction d'applications	Revues et études de transformation vers des plateformes low-code	Dépendance aux plateformes, gouvernance et intégration persistantes

La comparaison des travaux étudiés ne permet pas seulement d'identifier des contributions isolées ; elle met aussi en évidence des limites récurrentes qui traversent plusieurs propositions. Ces limites concernent notamment l'insuffisance des informations portées par BPMN, la couverture partielle des artefacts générés, la dépendance à certaines plateformes, ainsi que les difficultés de traçabilité et de régénération. Le tableau IV synthétise ces limites et en dégage les implications pour le framework à concevoir dans ce mémoire.

TABLE IV – *Limites transversales des travaux étudiés et implications pour la recherche*

Limite observée dans la littérature	**Conséquence pour la génération applicative**	**Implication pour l'approche à concevoir**
BPMN ne contient pas suffisamment d'informations applicatives.	Les formulaires, les données, les rôles, les mappings et les règles de visibilité ne peuvent pas être déduits de manière fiable du diagramme seul.	Prévoir une spécification complémentaire qui enrichit BPMN sans lui faire porter toute la description technique de l'application.
Les travaux génèrent souvent des fragments : interface, API ou prototype.	Certaines couches de l'application restent hors du périmètre de génération et doivent être développées séparément.	Viser une génération couvrant plusieurs couches du socle applicatif : interface, logique applicative, runtime, données et sécurité.
Certaines approches dépendent fortement d'une plateforme, d'un langage ou d'un framework.	Le résultat obtenu devient difficile à transposer ou à faire évoluer hors de l'environnement prévu initialement.	Séparer autant que possible les décisions métier des détails technologiques, au moyen de représentations intermédiaires et de règles de projection explicites.
La traçabilité entre modèles et fichiers générés reste souvent limitée.	Lorsqu'un fichier est modifié ou régénéré, il devient difficile de retrouver l'élément du modèle qui l'a produit.	Conserver des liens explicites entre les modèles sources, les représentations intermédiaires et les artefacts produits.
La cohabitation entre code généré et code manuel est difficile.	Les adaptations du développeur peuvent être perdues ou entrer en conflit avec une nouvelle génération.	Organiser dès le départ la séparation entre éléments générés, extensions manuelles et mécanismes de personnalisation.
La régénération après modification du modèle ou personnalisation du code est peu stabilisée.	L'application peut se désaligner du processus ou écraser silencieusement des fichiers déjà adaptés.	Prévoir une régénération contrôlée, capable d'identifier les fichiers suivis, les modifications locales et les conflits éventuels.
Les évaluations sont souvent limitées à un cas d'étude ou à un contexte spécifique.	La généralisation des résultats doit rester prudente et dépend du domaine, du modèle et de l'environnement technique.	Évaluer l'approche sur un cas suffisamment représentatif, tout en reconnaissant clairement les limites de généralisation.

II.5. Conclusion

Les travaux étudiés montrent que les modèles de processus peuvent guider l'exécution d'un workflow et soutenir la production d'interfaces, d'API, de prototypes ou de bases applicatives. Ils confirment aussi que BPMN constitue un point d'entrée pertinent pour représenter le fonctionnement métier. Cependant, ils révèlent plusieurs limites persistantes : le modèle BPMN doit être complété par des informations applicatives, la couverture générée reste souvent partielle, certaines solutions demeurent liées à une plateforme ou à un langage, la traçabilité modèle–code reste fragile et l'évolution du code généré demeure délicate.

Ces exigences structurent la conception du framework ProOWeb, présenté au chapitre suivant comme la solution proposée dans ce mémoire pour générer un socle applicatif web métier à partir de modèles BPMN enrichis.

III

CHAPITRE

APPROCHE D'INGÉNIERIE PROPOSÉE POUR LA GÉNÉRATION D'APPLICATIONS WEB PILOTÉES PAR BPMN

SOMMAIRE

III.1 - Introduction 42
III.2 - Vue d'ensemble de l'approche 43
III.3 - Architecture globale du framework proposé 44
III.4 - Modèle d'entrée enrichi : spécification spec-v1 46
III.5 - Règles de transformation et génération de l'application web 52
III.6 - Préservation du code personnalisé et synchronisation 60
III.7 - Conclusion 62

III.1. Introduction

Ce chapitre présente l'approche proposée pour générer une application web métier à partir d'un modèle BPMN complété par une spécification applicative. Cette approche est concrétisée dans ProOWeb, le framework développé dans ce travail.

L'objectif est de montrer comment l'approche conserve la traçabilité entre le processus et le code généré, tout en laissant au développeur la possibilité d'adapter l'application. Le chapitre présente successivement la vue d'ensemble, l'architecture, la spécification d'entrée, les transformations et les mécanismes de préservation du code personnalisé.

III.2. Vue d'ensemble de l'approche

III.2.1. Problème traité et principe général

L'approche proposée vise à générer une application web métier à partir d'un processus métier modélisé. ProOWeb doit fournir un socle applicatif que le développeur peut tester, adapter, versionner et maintenir.

Le choix de BPMN comme modèle d'entrée se justifie par sa capacité à représenter les activités, les événements, les passerelles et les flux d'un processus métier. Toutefois, BPMN ne décrit pas toujours les informations nécessaires à la génération d'une application web, telles que les formulaires, les droits d'accès, la persistance ou les règles d'affectation. Pour cette raison, ProOWeb repose sur un double modèle d'entrée : le fichier BPMN décrit le déroulement du processus, tandis que la spécification *spec-v1* fournit les détails applicatifs nécessaires.

Le fonctionnement général suit quatre étapes : analyser le modèle BPMN, lire la spécification *spec-v1*, construire un modèle applicatif intermédiaire, puis générer les artefacts liés à l'interface, à la logique métier, aux données, à la sécurité et au suivi du processus.

III.2.2. Positionnement dans le cycle MDA

La Model-Driven Architecture fournit un cadre pertinent pour comprendre la progression proposée. ProOWeb ne reprend pas ce cadre comme une classification abstraite ; il l'utilise pour clarifier les responsabilités de chaque niveau de la chaîne de génération.

TABLE V – *Positionnement de ProOWeb dans le cycle MDA*

Niveau	Contenu dans ProOWeb	Rôle dans la génération
PIM	BPMN et spécification *spec-v1*	Décrire le workflow, les rôles, les entrées, les sorties, la visibilité et les données sans dépendre directement du code cible.
PSM	Contrats intermédiaires, catalogues et registres	Stabiliser les informations dans une forme exploitable par les générateurs d'interface, de logique applicative et de gestion des données.
Code	Projet applicatif comprenant interface, logique métier, données, sécurité et configuration	Produire un socle applicatif fonctionnel, exécutable, versionnable et personnalisable.

III.2.3. Contribution méthodologique de l'approche

La contribution de ce travail se situe à l'intersection de trois préoccupations. La première est la modélisation du processus. Le BPMN reste la référence du workflow et permet de maintenir une continuité avec les pratiques de modélisation métier [54]. La deuxième est la génération d'une application web métier. Le framework ProOWeb ne se limite pas à produire des écrans ou des services isolés ; il organise un projet cible avec ses modules fonctionnels et techniques. La troisième est la maintenance du code généré. La régénération est traitée comme une opération contrôlée, accompagnée par un manifeste, des sauvegardes, des rapports et des mécanismes d'extension.

ProOWeb vise donc un résultat exploitable par une équipe de développement : un socle d'application web métier généré à partir du processus, traçable et ouvert à l'adaptation. Le framework automatise les parties décrites par ses modèles et ses gabarits ; il ne prétend ni déduire les règles absentes des sources ni supprimer le travail d'intégration du développeur.

III.3. Architecture globale du framework proposé

L'architecture de ProOWeb est organisée autour de quatre parties complémentaires. La première regroupe les modèles d'entrée. La deuxième correspond au noyau de génération du framework. La troisième représente l'application web produite. La quatrième concerne l'exploitation et l'évolution de cette application. Cette lecture par blocs permet de distinguer clairement ce qui est fourni par le concepteur, ce qui est traité par ProOWeb, ce qui est généré et ce qui est utilisé ensuite par les acteurs du système.

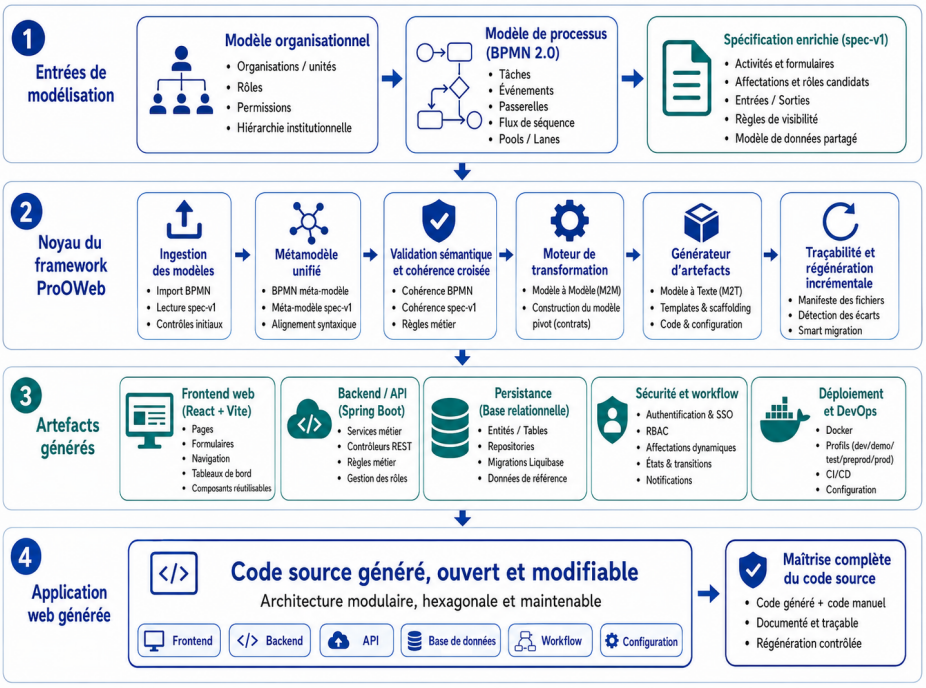


FIGURE 27 – *Architecture générale de ProOWeb*

III.3.1. Modèles d'entrée de ProOWeb

Le premier bloc de l'architecture constitue le point de départ de la génération. Il regroupe le modèle BPMN, la spécification *spec-v1*, les informations organisationnelles et la configuration du projet cible. Le modèle BPMN décrit le déroulement du processus à travers les activités, événements, passerelles et flux, tandis que la spécification *spec-v1* apporte les éléments nécessaires à l'application, notamment les rôles, les données, les règles d'affectation, de stockage et de visibilité. Les informations organisationnelles relient les tâches aux acteurs concernés, alors que la configuration précise les options générales du projet à générer. Ce bloc joue ainsi le rôle de source de vérité : ProOWeb y lit, valide et transforme les informations métier afin de préserver la traçabilité entre le modèle initial et le code produit.

III.3.2. Noyau de génération du framework

Ce bloc permet de transformer les modèles d'entrée en artefacts générables à travers plusieurs traitements : ingestion du BPMN, enrichissement par la spécification *spec-v1*, validation croisée, construction du modèle pivot et génération des fichiers applicatifs. L'ingestion extrait les activités, événements, passerelles et flux du processus, puis l'enrichissement associe à chaque activité les rôles, données, règles de visibilité et comportements définis dans la spécification. La validation vérifie ensuite la cohérence entre le BPMN et *spec-v1*, afin d'éviter une applica-

tion générée fonctionnellement incorrecte. Après cette étape, ProOWeb construit un modèle pivot composé de contrats intermédiaires, tels que le *runtime contract*, le *data contract*, les catalogues d'interface et le registre des processus. Ces contrats structurent les transformations modèle-à-modèle et modèle-à-texte nécessaires à la production du code source.

III.3.3. Application web générée

Ici, il ne s'agit pas d'un simple assemblage de fragments, mais d'un projet web métier structuré, exécutable et exploitable par les acteurs concernés. L'application générée comprend les vues de traitement, les listes de tâches, les formulaires, les interfaces de suivi et les espaces de supervision. Elle intègre également les services nécessaires au démarrage d'un processus, à l'exécution des activités, à la gestion des transitions, à la persistance des données et au contrôle des droits d'accès. Les données peuvent être rattachées à une instance de processus ou conservées comme informations partagées, selon les règles définies dans *spec-v1*. Ainsi, ProOWeb produit une base applicative complète, reliant interface, logique métier, sécurité, données, exécution du workflow et configuration du projet cible.

III.3.4. Exploitation et évolution de l'application

Après la génération, l'application est utilisée par les acteurs métier pour exécuter les tâches, par les superviseurs pour suivre les instances, par les administrateurs pour gérer les paramètres et par les développeurs pour adapter le code lorsque des besoins spécifiques apparaissent. Cette dimension évolutive est essentielle, car un processus, une règle d'affectation, un champ ou une interface peuvent être modifiés au cours du temps. ProOWeb prend donc en compte la régénération à travers le suivi des fichiers produits, la détection des modifications manuelles, la création de sauvegardes et la production de rapports de migration. L'architecture forme ainsi une boucle itérative reliant les modèles, le code généré et les adaptations réalisées.

III.4. Modèle d'entrée enrichi : spécification spec-v1

La spécification *spec-v1* constitue le complément applicatif du modèle BPMN. Elle rend explicites les informations nécessaires à la génération d'un socle applicatif web métier : rôles autorisés, supervision, stratégies d'affectation, données d'entrée, sorties, règles de visibilité, stockage et comportement des activités automatiques.

III.4.1. Grammaire de spec-v1

Dans ProOWeb, la grammaire ne décrit pas l'ordre d'exécution du processus : cette information reste portée par BPMN [55]. Elle formalise les renseignements ajoutés par *spec-v1* pour rendre le processus projetable vers un socle applicatif.

Le modèle grammatical attribué d'une spécification est défini par :

$$G_{\text{spec}} = (p, v, R, S, M, A, D, C). \tag{III.1}$$

Les composantes de ce modèle ont la signification suivante :

— p est l'identifiant du processus BPMN enrichi. Dans le format d'import du cas d'étude, il correspond à `processId` ; ProOWeb le conserve ensuite comme clé de modèle dans le catalogue du workspace ;
— v est la version de la spécification. Le langage étudié dans ce mémoire impose $v = 1$;
— R est l'ensemble fini des rôles mentionnés dans la spécification : rôles de démarrage, de supervision, d'affectation et de visibilité ;
— $S = (R_s, A_s, b_s)$ décrit le démarrage. $R_s \subseteq R$ contient les rôles autorisés à créer une instance, A_s les identifiants des activités initiales et b_s indique si un démarrage automatique est permis en l'absence d'une entrée manuelle ;
— $M \subseteq R$ est l'ensemble des rôles autorisés à superviser le processus ;
— $A = \{a_1, \ldots, a_n\}$ est l'ensemble fini des activités enrichies. Chaque a_i porte l'identifiant d'une activité présente dans le BPMN ;
— $D = \{d_1, \ldots, d_m\}$ est l'ensemble fini des entités partagées déclarées par le processus ;
— C est l'ensemble des contraintes de validité contrôlées avant la construction des contrats intermédiaires.

L'activité constitue l'unité principale d'enrichissement de *spec-v1*. Elle est représentée par le tuple suivant :

$$a_i = (id_i, t_i, R_i, \alpha_i, I_i, O_i, V_i, X_i). \tag{III.2}$$

Dans cette représentation, id_i est l'identifiant BPMN de l'activité. Son type t_i prend la valeur `MANUAL` ou `AUTOMATIC`. L'ensemble R_i contient ses rôles candidats et α_i sa politique d'affectation. I_i regroupe les sources d'entrée. La sortie $O_i = (s_i, \mu_i)$ associe une stratégie de stockage s_i à un ensemble de mappings μ_i. La visibilité $V_i = (R_i^a, R_i^d)$ distingue les rôles autorisés à voir l'activité de ceux qui peuvent consulter ses données. Enfin, X_i décrit le gestionnaire et le mode de

déclenchement lorsque l'activité est automatique ; il reste vide pour une activité manuelle.

Une entité partagée $d_j = (k_j, F_j)$ possède une clé k_j et un ensemble de champs F_j. Chaque champ est décrit par son nom, son type et son caractère obligatoire. Cette définition suffit pour que le contrat de données prépare les structures persistantes et vérifie les cibles des mappings.

Les contraintes C donnent sa portée opérationnelle au modèle grammatical. Elles imposent notamment que p corresponde au processus BPMN importé, que $\{id_1, \ldots, id_n\}$ soit inclus dans l'ensemble des activités du BPMN et que les rôles employés appartiennent au référentiel retenu. De plus, une activité manuelle doit posséder au moins un rôle candidat, un mapping doit viser une donnée déclarée et une activité automatique doit référencer un gestionnaire connu. Une violation bloquante empêche la promotion des contrats. Ainsi, *spec-v1* reste une spécification complémentaire à BPMN. Le listing suivant en présente une structure JSON simplifiée.

```json
{
  "processId": "processus_1",
  "schemaVersion": 1,
  "start": {
    "startableByRoles": ["ROLE_3"],
    "startActivities": ["activité_1"]
  },
  "monitors": {
    "monitorRoles": ["ROLE_1", "ROLE_2"]
  },
  "activities": {
    "activité_1": {
      "activityType": "MANUAL",
      "candidateRoles": ["ROLE_3"],
      "assignment": {
        "mode": "AUTOMATIC",
        "strategy": "ROLE_QUEUE"
      },
      "input": {},
      "output": {},
      "visibility": {}
    }
  },
  "sharedData": {
    "entities": []
  }
}
```

Listing III.1 – *Structure simplifiée de la spécification spec-v1*

III.4.2. Propriétés de la grammaire spec-v1

Ces propriétés précisent dans quelle mesure *spec-v1* remplit son rôle de complément à BPMN. Elles portent sur la qualité de la spécification en entrée et sur son utilité pour construire les contrats intermédiaires.

Le tableau VI résume ces propriétés, leur signification et leur intérêt dans la chaîne de génération.

TABLE VI – *Propriétés de la grammaire spec-v1*

Propriété	Signification	Intérêt pour ProOWeb
Complétude relative	La spécification complète BPMN avec les informations requises pour la génération, sans chercher à décrire tout le domaine métier.	Les rôles, formulaires, données et politiques deviennent projetables.
Cohérence	Les activités, rôles, données et règles doivent être compatibles avec le BPMN et le référentiel de rôles.	Les contradictions sont détectées avant leur propagation dans plusieurs fichiers.
Vérifiabilité	Les règles syntaxiques et sémantiques peuvent être évaluées avant la génération.	ProOWeb peut produire un rapport localisé et bloquer une entrée invalide.
Traçabilité	Une activité enrichie conserve l'identifiant de l'activité BPMN correspondante.	Cet identifiant est repris dans les contrats et les ressources versionnées.
Modularité	Chaque processus peut posséder sa propre spécification dans un workspace commun.	Les processus évoluent séparément tout en partageant catalogues et conventions.
Extensibilité	La version de schéma permet d'ajouter des champs, des types ou des stratégies.	Le langage peut accompagner de nouvelles règles de projection sans modifier son principe général.

III.4.3. Métadonnées des activités

Les métadonnées d'activité indiquent comment une activité BPMN doit être prise en charge dans l'application. Une activité manuelle est destinée à être traitée par un utilisateur. Elle doit donc préciser les rôles candidats, la stratégie d'affectation, les données visibles, les sorties attendues et les règles de complétion. Une activité automatique est exécutée par le système. Elle doit alors indiquer le gestionnaire à invoquer, son type, son mode de déclenchement et sa configuration. L'extrait JSON montre la structure d'une activité enrichie.

```
"activite_1": {
  "activityType": "MANUAL",
  "candidateRoles": ["ROLE_1"],
  "assignment": {
    "mode": "AUTOMATIC",
    "strategy": "ROLE_QUEUE"
  },
```

```
  "input": {
    "sources": [
      {"sourceType": "PROCESS_CONTEXT", "sourceRef": "process",
       "mappings": []}
    ]
  },
  "output": {
    "storage": "BOTH",
    "mappings": []
  },
  "visibility": {
    "activityViewerRoles": ["ROLE_1", "ROLE_2"],
    "dataViewerRoles": ["ROLE_1"]
  }
}
```

Listing III.2 – *Activité enrichie dans spec-v1*

Cet exemple montre le passage du processus vers l'application. L'identifiant `valider_demande` renvoie à une activité BPMN. Les rôles candidats orientent l'affectation de la tâche. Les sources d'entrée indiquent les données disponibles pour le formulaire. La stratégie de stockage précise où seront conservées les sorties. Les règles de visibilité distinguent les acteurs qui peuvent voir l'activité de ceux qui peuvent accéder aux données.

III.4.4. Affectation des tâches et sécurité

L'affectation des tâches est un point central dans une application web métier. Une tâche doit être proposée aux bons utilisateurs, selon les responsabilités prévues par l'organisation. Dans ProOWeb, cette dimension est décrite par les rôles candidats, le mode d'affectation et la stratégie d'assignation.

TABLE VII – *Stratégies d'affectation*

Stratégie	Interprétation dans ProOWeb
`ROLE_QUEUE`	La tâche est visible dans une file associée aux rôles candidats.
`UNIT_MEMBERS`	La tâche est proposée aux membres d'une unité organisationnelle admissible.
`SUPERVISOR_ONLY`	La tâche est réservée aux rôles de supervision.
`SUPERVISOR_THEN_ANCESTORS`	La tâche peut être escaladée dans une chaîne hiérarchique lorsque la règle locale ne suffit pas.

III.4.5. Entrées, sorties et données partagées

Pour une activité BPMN enrichie, les entrées indiquent d'où viennent les données nécessaires à son exécution. ProOWeb distingue le contexte de l'instance, les résultats d'une activité antérieure, les données partagées et les informations obtenues auprès d'un service backend ou externe. Le champ `sourceRef` identifie la source précise et les mappings indiquent comment ses valeurs alimentent les données locales de l'activité.

Les sorties décrivent les données produites lorsque l'activité est achevée. Un mapping associe un chemin produit à un chemin cible. La stratégie `INSTANCE` conserve la valeur dans l'instance courante, `SHARED` la dirige vers une entité partagée et `BOTH` applique les deux formes de stockage. La sortie d'une activité peut ainsi devenir l'entrée d'une activité ultérieure.

Le bloc `sharedData` définit les entités et les champs accessibles à plusieurs activités du modèle. Il constitue l'espace d'échange durable du processus : une activité peut y publier une donnée par un mapping de sortie, puis une autre peut la lire en déclarant une source de type `SHARED_DATA`. Le tableau VIII résume ces mécanismes de circulation.

TABLE VIII – *Entrées, sorties et données partagées d'une activité BPMN enrichie*

Élément de spec-v1	Signification pour l'activité	Circulation de la donnée
`PROCESS_CONTEXT`	Données déjà présentes dans l'instance du processus.	Du contexte courant vers l'entrée de l'activité.
`PREVIOUS_ACTIVITY`	Données produites par une activité antérieure désignée par `sourceRef`.	De la sortie de l'activité source vers l'entrée de l'activité courante.
`SHARED_DATA`	Valeurs lues dans une entité partagée déclarée.	De l'espace partagé vers une ou plusieurs activités.
Service backend ou externe	Données résolues par un service applicatif ou un système tiers.	Du service invoqué vers l'entrée de l'activité.
`output.mappings`	Correspondances entre les chemins produits et leurs cibles.	De la sortie de l'activité vers l'instance ou une entité partagée.
`output.storage`	Choix entre `INSTANCE`, `SHARED` et `BOTH`.	Détermine où les valeurs produites sont conservées.
`sharedData.entities`	Schéma des données métier communes aux activités.	Une activité publie la valeur ; d'autres activités peuvent ensuite la consulter.

III.4.6. Règles de visibilité

La visibilité ne se réduit pas à l'affectation. Un utilisateur peut être autorisé à consulter une instance sans être autorisé à traiter la tâche courante. De même, un superviseur peut voir l'état d'un processus sans accéder à toutes les données sensibles produites par les activités. ProOWeb distingue donc la visibilité des activités et la visibilité des données.

Cette distinction permet de générer des filtres plus fins côté services applicatifs et des interfaces plus adaptées côté utilisateur. Les services applicatifs peuvent masquer certaines tâches ou certaines données selon le rôle de l'utilisateur. Les composants d'interface peuvent également adapter les informations affichées sans exposer inutilement des données sensibles.

III.5. Règles de transformation et génération de l'application web

Les règles de transformation déterminent la manière dont le double modèle d'entrée devient une application web métier. Dans ProOWeb, la transformation suit deux moments principaux. La transformation modèle-à-modèle construit le modèle applicatif intermédiaire. La transformation modèle-à-texte produit les fichiers du projet cible. Ce découpage permet de vérifier les contrats avant l'écriture du code.

III.5.1. Validation avant génération

La validation est une étape indispensable. Une erreur dans les modèles d'entrée peut se propager dans plusieurs fichiers générés : point d'accès applicatif, composant d'interface, transition, structure de données ou règle de sécurité. Le framework doit donc refuser ou signaler les situations ambiguës avant de produire le projet cible.

Les contrôles portent sur quatre dimensions. La dimension structurelle vérifie que les activités enrichies existent dans le BPMN et que les transitions forment un graphe exploitable. La dimension applicative vérifie que les activités manuelles disposent d'informations suffisantes pour être traitées par un utilisateur. La dimension données vérifie que les entrées, sorties et mappings référencent des structures connues. La dimension sécurité vérifie la cohérence des rôles utilisés pour l'exécution, la supervision et la visibilité.

TABLE IX – *Contrôles réalisés avant génération*

Dimension	Règle contrôlée	Risque évité
Structure BPMN	Toute activité enrichie doit exister dans le modèle BPMN.	Artefacts sans activité correspondante.
Sécurité	Une tâche manuelle doit posséder au moins un rôle candidat.	Tâche exposée sans responsabilité claire.
Données	Un mapping de sortie doit cibler une donnée connue.	Perte ou mauvaise persistance des données.
Workflow	Une transition générée doit correspondre à un chemin possible.	Divergence entre modèle et runtime.
Interface	Les champs affichés doivent provenir d'une source déclarée.	Formulaire incohérent avec le contexte.

Ces contrôles doivent produire des diagnostics compréhensibles. L'utilisateur du framework doit pouvoir identifier l'activité, la règle ou la donnée responsable de l'erreur.

III.5.2. Transformation modèle-à-modèle

La transformation M2M reçoit le modèle BPMN, *spec-v1*, le référentiel de rôles et la configuration du projet. ProOWeb extrait les activités, les événements, les passerelles, les flux, les pools et les couloirs du BPMN, puis lit les enrichissements applicatifs. Il contrôle l'identité du processus, l'existence des activités référencées, les rôles, les affectations, les entrées, les sorties, les mappings et la visibilité. Une erreur bloquante interrompt la transformation avant la production d'un modèle exploitable.

Après validation, chaque activité est normalisée avec son type, sa politique d'affectation, ses sources de données, ses sorties et son modèle de formulaire. Le graphe BPMN fournit ensuite les transitions du `RuntimeContract`, tandis que les entités partagées et les mappings alimentent le `DataContract`. Les identifiants du modèle, de sa version et des activités sont conservés pour assurer la traçabilité vers les futurs artefacts.

III.5.2.1. Algorithme de transformation M2M

L'algorithme 1 reçoit le modèle BPMN, la spécification *spec-v1*, le référentiel de rôles et la configuration du projet. Il décrit le passage de ces sources vers le modèle pivot de ProOWeb et produit simultanément un rapport de validation.

Algorithme 1 : TransformerM2M(B, S, R, C)

```
1  Entrées :
2      B : modèle BPMN
3      S : spécification spec-v1
4      R : référentiel de rôles
5      C : configuration du projet
6  Sorties :
7      P : modèle pivot ProOWeb
8      V : rapport de validation
9  Début
10     V ← creerRapportValidation()
11     E ← extraireBPMN(B,
12             {activites, evenements, passerelles, flux, pools, lanes})
13     D ← lireBlocsSpecV1(S,
14             {processId, schemaVersion, start, monitors,
15              activites, sharedData})
16     verifierIdentiteProcessus(D.processId, E.processus, C, V)
17     verifierExistenceActivites(D.activities, E.activites, V)
18     verifierRolesEtAffectations(D, R, V)
19     verifierEntreesSortiesMappings(D, V)
20     verifierVisibilite(D, R, V)
21     Si V.contientErreurBloquante() Alors
22         retourner (INDEFINI, V)
23     FinSi
24     P ← creerModelePivot(C)
25     P.processModel ← creerProcessModel(E.processus, E.pools, E.lanes)
26     Pour chaque activite a de E.activites Faire
27         enrichissement ← chercherEnrichissement(a.id, D.activities)
28         activitePivot ← creerActivityModel(a, enrichissement)
29         activitePivot.rolePolicy ← construireRolePolicy(enrichissement, R)
30         activitePivot.formModel ← construireFormModel(
31                                       enrichissement.input,
32                                       enrichissement.output)
33         ajouter(P.processModel, activitePivot)
34     FinPour
35     P.runtimeContract ← construireRuntimeContract(
36                             P.processModel, E.evenements,
37                             E.passerelles, E.flux, D.start)
38     P.dataContract ← construireDataContract(
39                         D.sharedData, D.activities.mappings)
40     P.artifactTraces ← construireArtifactTraces(
41                         B, S, P.processModel,
42                         P.runtimeContract, P.dataContract)
43     ajouterDiagnosticsNormalisation(V, P)
44     retourner (P, V)
45 Fin
```

FIGURE 28 – *Algorithme de transformation modèle-à-modèle BPMN/spec-v1 vers le modèle pivot*

Son fonctionnement s'organise autour de trois opérations. ProOWeb extrait d'abord la structure du processus et lit les blocs utiles de *spec-v1*. Il vérifie ensuite l'identité du processus, les activités enrichies, les rôles, les mappings et la visibilité ; une erreur bloquante interrompt la transformation. Les éléments validés sont enfin normalisés dans le modèle de processus, les politiques d'affectation et les for-

mulaires, puis dans les contrats runtime et data. Les traces d'artefacts conservent l'origine des éléments destinés à la génération. Le pivot désigne ici cet ensemble cohérent de contrats, et non nécessairement un fichier unique. Dans le prototype, le contrôle des codes de rôle est effectif, mais leur confrontation exhaustive à un référentiel organisationnel externe reste à consolider.

III.5.2.2. Propriétés de l'algorithme de transformation M2M

L'étude de l'algorithme M2M porte sur les garanties attendues lors du passage du couple BPMN–*spec-v1* aux contrats intermédiaires. Le tableau X relie chaque propriété aux mécanismes effectivement mobilisés par ProOWeb.

TABLE X – *Propriétés de l'algorithme de transformation M2M*

Propriété	Signification	Justification dans ProOWeb
Terminaison	La transformation doit finir pour toute entrée finie.	Les traitements parcourent des ensembles finis d'activités, de flux, de rôles, de sources et de mappings ; aucune récursion non bornée n'est requise.
Déterminisme	Des entrées normalisées identiques doivent produire les mêmes contrats.	Les identifiants d'activités, les rôles, les transitions et les entités sont triés avant la construction des `RuntimeContract` et `DataContract`.
Cohérence	Le pivot ne doit contenir que des éléments compatibles entre eux.	La correspondance avec le BPMN, les types d'activités, les affectations, les sources et les mappings est contrôlée avant la construction des contrats.
Sûreté	Une erreur bloquante ne doit pas conduire à un pivot exploitable.	Le rapport de validation marque la spécification comme invalide et empêche la promotion des contrats lorsque des erreurs subsistent.
Traçabilité	Les éléments intermédiaires doivent rester reliés à leurs sources.	La clé et la version du modèle, les identifiants BPMN et les chemins des ressources sont conservés dans les contrats et les catalogues.
Extensibilité	Une nouvelle règle ne doit pas imposer la réécriture complète de la transformation.	Les fonctions de normalisation et les constructeurs de contrats sont séparés ; une extension reste localisée tant que la structure générale du pivot est préservée.

Construction du modèle applicatif intermédiaire

Le modèle applicatif intermédiaire regroupe uniquement les informations nécessaires à la génération : déroulement du processus, rôles, données, formulaires et traces des artefacts produits. Il sert de contrat entre l'analyse du BPMN, l'enrichissement *spec-v1* et les gabarits de génération.

TABLE XI – *Structures principales du modèle pivot ProOWeb*

Structure conservée	Utilité dans ProOWeb
`ProcessModel`	Garde la référence au processus BPMN, à ses activités et à ses transitions.
`ActivityModel`	Transforme chaque activité BPMN enrichie en unité exploitable par l'application.
`RolePolicy`	Regroupe les rôles, règles d'affectation, droits d'accès et restrictions de visibilité.
`FormModel`	Décrit les champs affichés, les sorties produites et les validations des formulaires.
`RuntimeContract`	Fournit au moteur d'exécution les états, transitions et actions disponibles.
`DataContract`	Prépare les entités, champs, mappings et règles de stockage des données.
`ArtifactTrace`	Relie les éléments du modèle aux fichiers générés afin de contrôler les régénérations.

III.5.3. Transformation modèle-à-texte

La transformation M2T reçoit le modèle pivot, les gabarits, la configuration et le workspace cible. Elle projette d'abord les contrats runtime et data, puis produit les ressources backend, frontend, de sécurité, de configuration et de test. Les gabarits traduisent les décisions déjà stabilisées dans le pivot : ils ne doivent introduire ni rôle, ni transition, ni règle métier absente des contrats.

Les artefacts sont ordonnés par chemin avant leur écriture afin de conserver une structure reproductible. Pour chaque fichier géré, ProOWeb calcule une empreinte SHA-256 et enregistre son chemin, sa catégorie et ses propriétaires logiques dans le manifeste. Lorsqu'un fichier existant a été personnalisé, les services de déploiement et de migration détectent le conflit et créent, selon la politique appliquée, une sauvegarde avant tout remplacement contrôlé.

III.5.3.1. Algorithme de génération Model-to-Text

L'algorithme 2 présente la projection du modèle pivot vers le workspace cible. Il prend en compte les gabarits et la configuration du projet, puis retourne les artefacts produits et le manifeste qui permet de les suivre.

Algorithme 2: GenererM2T(P, T, C, W)

Entrées :
P : modèle pivot ProOWeb
T : ensemble de gabarits de generation
C : configuration du projet
W : workspace cible

Sorties :
A : artefacts applicatifs generes
M : manifeste de generation

```
1  Début
2      W ← initialiserOuChargerWorkspace(W, C)
3      ancienManifeste ← chargerManifesteSiPresent(W)
4      A ← ensembleVide()
5      M ← creerManifeste(C)
6      ajouter(A, genererRessourcesRuntime(P.runtimeContract, T, C))
7      ajouter(A, genererRessourcesDonnees(P.dataContract, T, C))
8      ajouter(A, genererBackend(P,
                   {entites, services, controleurs, registres,
                   gestionnaires, reglesApplicatives}, T, C))
9      ajouter(A, genererFrontend(P,
                   {pages, formulaires, tableauxDeBord,
                   clientsAPI, vuesTraitement}, T, C))
10     ajouter(A, genererAccesEtVisibilite(P, T, C))
11     ajouter(A, genererConfigurationScriptsEtTests(P, T, C))
12     Pour chaque artefact a de trierParChemin(A) Faire
13         cible ← resoudreChemin(W, a.chemin)
14         etat ← classifierFichier(cible, ancienManifeste)
15         Si etat = PERSONNALISE Alors
16             signalerConflitSansEcraser(cible, a)
17         Sinon
18             ecrireFichier(cible, a.contenu)
19             empreinte ← calculerEmpreinte(a.contenu)
20             enregistrer(M,
                   {chemin: a.chemin, categorie: a.categorie,
                   proprietaire: a.proprietaireLogique,
                   source: a.artifactTrace, hash: empreinte})
21         FinSi
22     FinPour
23     sauvegarderManifeste(W, M)
24     retourner (A, M)
25 Fin
```

FIGURE 29 – *Algorithme de transformation Model-to-Text vers le socle applicatif*

L'algorithme initialise le workspace, projette les contrats runtime et data, puis génère les ressources backend, frontend, de sécurité, de configuration et de test. Les artefacts sont traités selon un ordre stable afin de conserver une structure reproductible. Pour chaque fichier effectivement écrit, ProOWeb calcule une empreinte et enregistre son chemin, sa catégorie, son propriétaire logique dans le manifeste. Le lien avec la source est déduit de ces propriétaires et des chemins versionnés ; le

manifeste actuel ne sérialise pas un champ `ArtifactTrace` autonome. Une personnalisation détectée n'est jamais remplacée silencieusement : selon la politique de migration, ProOWeb la signale ou crée une sauvegarde avant remplacement. Cette protection mobilise les services de déploiement et de migration en complément du rendu des gabarits. La transformation M2T reste donc une projection : elle matérialise les décisions du pivot sans introduire de rôle, de transition ou de règle métier absente des contrats.

III.5.3.2. Propriétés de l'algorithme de transformation M2T

Pour la transformation M2T, l'analyse concerne la production et le suivi des fichiers du workspace. Le tableau XII précise les garanties apportées par les gabarits, le manifeste et les mécanismes de déploiement contrôlé.

TABLE XII – *Propriétés de l'algorithme de transformation M2T*

Propriété	Signification	Justification dans ProOWeb
Terminaison	La génération doit finir lorsque le pivot et les gabarits sont finis.	ProOWeb parcourt un nombre borné de contrats, d'activités, d'entités, de gabarits et de chemins de fichiers.
Déterminisme	Les mêmes entrées doivent produire la même structure applicative.	Les artefacts sont ordonnés par chemin et les mêmes gabarits produisent les mêmes contenus. Les dates techniques du manifeste peuvent néanmoins varier entre deux exécutions.
Cohérence	Chaque fichier doit matérialiser une décision présente dans le pivot.	Les ressources runtime, data, backend, frontend et de sécurité sont produites à partir des contrats validés, sans redéfinir les règles métier dans les gabarits.
Traçabilité	Un fichier généré doit rester identifiable après son écriture.	Le manifeste conserve son chemin, sa catégorie, ses propriétaires logiques et son empreinte SHA-256 ; les ressources de processus sont également versionnées.
Préservation contrôlée	Une personnalisation ne doit pas être perdue silencieusement.	Les empreintes permettent de détecter un fichier modifié ; ProOWeb le signale et crée une sauvegarde avant tout remplacement autorisé par la politique de migration.
Maintenabilité	Le résultat doit pouvoir être repris par un développeur.	Les artefacts sont organisés par couches et responsabilités dans une arborescence backend/frontend explicite.
Extensibilité	La génération doit accepter de nouvelles projections.	Des gabarits et des overrides peuvent être ajoutés sans modifier la sémantique des contrats intermédiaires.

III.5.4. Règles de projection vers les artefacts

Les règles de projection définissent comment chaque élément du modèle pivot est traduit dans l'application cible. Elles ne sont pas de simples conventions de nommage. Elles indiquent la couche de destination et la responsabilité de l'artefact produit.

TABLE XIII – *Règles de projection vers l'application générée*

Élément du modèle pivot	**Artefact généré**	**Couche cible**
Activité manuelle	Formulaire, page d'activité, structure d'entrée	Interface utilisateur et couche applicative.
Activité automatique	Opération système ou service applicatif	Couche applicative.
Rôle candidat	Règle d'accès et filtre de tâche	Sécurité, services applicatifs et interface.
Transition du workflow	Méthode de progression et changement d'état	Runtime du processus.
Entité partagée	Structure métier, stockage, migration	Domaine et persistance.
Mapping de sortie	Service de sauvegarde et projection cible	Application et persistance.

Cette projection maintient la cohérence entre le modèle et le projet produit. Le générateur ne crée pas seulement des fichiers ; il garde le lien entre chaque élément généré et le besoin métier qui l'a rendu nécessaire. Ce lien rend le code plus compréhensible lorsqu'il faut l'adapter ou le régénérer.

III.5.5. Traçabilité de bout en bout

La traçabilité est illustrée ici sur l'activité BPMN `activité_1`. L'intérêt n'est pas de détailler séparément chaque couche technique, mais de montrer comment une intention métier reste reconnaissable jusqu'aux éléments produits par ProO-Web.

TABLE XIV – *Traçabilité d'une activité BPMN vers les artefacts générés*

Niveau suivi	Interprétation dans ProOWeb
BPMN	La tâche `activité_1` donne le point de passage métier et ses liens avec le reste du processus.
spec-v1	L'activité reçoit les précisions absentes du diagramme : rôle autorisé, données utiles, sorties attendues et visibilité.
Modèle pivot	ProOWeb rassemble ces informations dans une représentation stable, utilisable par les règles de génération.
Artefacts générés	Les fichiers produits matérialisent la même activité : écran de traitement, action de validation, contrôle d'accès et mise à jour des données.
Manifeste	Les fichiers rattachés à cette activité restent identifiables afin de maîtriser les futures régénérations.

III.6. Préservation du code personnalisé et synchronisation

Une difficulté importante de la génération d'application concerne l'évolution du projet après une première génération. Dans un contexte réel, les développeurs ajoutent des règles particulières, adaptent les interfaces, complètent des services ou intègrent des dépendances spécifiques. Si chaque nouvelle génération écrase ces modifications, le framework devient difficile à utiliser. ProOWeb traite donc la régénération comme une opération d'ingénierie, et non comme un simple remplacement de fichiers.

Cette préoccupation est bien identifiée dans les travaux sur la cohabitation entre code généré et code écrit à la main. Greifenberg et al. comparent plusieurs mécanismes d'intégration et montrent que le choix d'une stratégie influence directement la maintenabilité des systèmes générés [44].

III.6.1. Problème de la régénération

La régénération apparaît lorsqu'un processus évolue, lorsqu'une activité est ajoutée, lorsqu'une règle de données change. Elle est nécessaire pour maintenir l'application alignée sur les modèles. Elle est également risquée, car elle peut remplacer des fichiers modifiés par les développeurs.

Le problème ne se limite pas à la perte de code. Une régénération non contrôlée peut produire des conflits de fichiers, des incohérences entre la partie applicative et l'interface, la disparition de règles spécifiques, un changement non documenté dans la sécurité ou une rupture des tests. ProOWeb distingue donc les fichiers gérés

par le générateur et les fichiers non gérés. Les fichiers gérés sont suivis dans le manifeste `.prooweb-managed.json`. Les fichiers non gérés restent hors du contrôle du générateur.

III.6.2. Manifeste et contrat de propriété

Le manifeste `.prooweb-managed.json` décrit les fichiers produits par ProOWeb. Pour chaque fichier, il peut conserver le chemin, la catégorie, les propriétaires logiques et une empreinte. Cette information permet de savoir si un fichier correspond toujours à la dernière version générée ou s'il a été modifié manuellement.

Ce manifeste définit un contrat de propriété. Un fichier généré et non modifié peut être remplacé lors d'une nouvelle génération. Un fichier généré mais modifié doit être traité avec prudence. Un fichier non géré ne doit pas être écrasé. Cette règle donne au développeur une garantie pratique : ProOWeb ne considère pas le projet cible comme un répertoire jetable.

III.6.3. Préservation côté logique applicative

Côté logique applicative, la préservation est particulièrement importante, car le code généré porte les API, la sécurité, la persistance et le runtime. ProOWeb limite le risque en produisant une architecture modulaire. Les éléments issus du modèle sont générés dans des couches précises, tandis que les adaptations locales peuvent être placées dans des modules ou points d'extension séparés.

Cette stratégie rejoint le principe du *Generation Gap Pattern*, souvent utilisé pour séparer les classes générées des classes écrites manuellement [44]. Le générateur peut produire un squelette ou une classe de base, tandis que le développeur ajoute un comportement spécifique dans une classe séparée. Lors d'une régénération, la partie contrôlée par ProOWeb peut évoluer sans supprimer les extensions manuelles.

La préservation ne doit toutefois pas servir à contourner le modèle. Si une modification concerne le workflow, les rôles, les données structurantes ou les règles de visibilité, elle devrait être reportée dans le BPMN ou dans *spec-v1*. Les extensions manuelles doivent plutôt couvrir les détails d'intégration, les règles particulières ou les adaptations difficiles à généraliser.

III.6.4. Préservation côté interface utilisateur

Côté interface utilisateur, la personnalisation concerne souvent les libellés, les parcours utilisateur, les composants visuels, les contrôles d'ergonomie et certaines intégrations. ProOWeb génère les composants liés au processus, mais l'équipe doit

pouvoir adapter l'expérience utilisateur sans perdre ses modifications lors d'un redéploiement.

La stratégie retenue repose sur la composition, la configuration et les overrides de templates. Les composants générés peuvent être encapsulés par des composants propres au projet. Les règles d'override déclarées peuvent remplacer, préfixer, suffixer ou modifier un bloc de contenu généré. Cette personnalisation est réappliquée lors des générations suivantes, ce qui réduit les modifications répétées directement dans les fichiers produits.

III.6.5. Cycle de synchronisation itératif

Le cycle de synchronisation décrit la manière dont ProOWeb accompagne les évolutions successives du modèle. Une première génération produit l'application de base. Les développeurs peuvent ensuite ajouter des extensions, adapter certains modules ou déclarer des overrides. Lorsque le BPMN ou *spec-v1* évolue, ProOWeb reconstruit un snapshot de génération, compare l'état attendu avec l'état du projet, applique les règles de migration et produit un rapport.

Ce cycle transforme la génération en activité continue. L'application n'est pas simplement créée une fois ; elle peut être régénérée, migrée et adaptée. La synchronisation repose sur trois traces : le manifeste des fichiers gérés, les sauvegardes créées lors des conflits et les rapports de migration. Ces éléments permettent de comprendre ce qui a changé et de revenir sur une modification si nécessaire.

TABLE XV – *Mécanismes de préservation et de synchronisation dans ProOWeb*

Mécanisme	**Rôle**	**Effet attendu**
Manifeste	Suivre les fichiers générés, leurs catégories et leurs empreintes.	Identifier les fichiers que ProOWeb peut gérer.
Migration contrôlée	Comparer l'état attendu et l'état réel du projet.	Éviter l'écrasement silencieux du code personnalisé.
Sauvegardes	Copier les fichiers conflictuels avant remplacement.	Permettre la récupération des adaptations manuelles.
Template overrides	Réappliquer des personnalisations déclarées.	Réduire les modifications manuelles répétées.
Rapports	Décrire les créations, mises à jour, conflits et sauvegardes.	Rendre l'évolution auditable et compréhensible.

III.7. Conclusion

Ce chapitre a présenté l'approche proposée et sa concrétisation dans le framework ProOWeb, depuis les modèles d'entrée jusqu'aux artefacts générés. Le BPMN décrit le déroulement du processus, *spec-v1* apporte les informations applicatives nécessaires et le modèle pivot organise ces données avant la génération.

L'approche met l'accent sur deux points : maintenir la traçabilité entre le modèle et le code, puis permettre l'évolution contrôlée de l'application générée. Le chapitre suivant présente la mise en œuvre technique de cette proposition et son application à un cas d'étude.

IV

CHAPITRE

MISE EN ŒUVRE ET ÉVALUATION DE PROOWEB

SOMMAIRE

IV.1 - Introduction 64
IV.2 - Environnement de mise en œuvre 65
IV.3 - Cas d'étude organisationnel 67
IV.4 - Déroulement complet du processus de commande client 69
IV.5 - Discussion et positionnement 78
IV.6 - Conclusion 79

IV.1. Introduction

Le chapitre précédent a présenté le framework proposé pour générer une application web métier à partir d'un modèle BPMN complété par une spécification applicative. Il a notamment défini l'architecture générale de ProOWeb, le rôle de *spec-v1*, les transformations modèle-à-modèle et modèle-à-texte, ainsi que les mécanismes de préservation du code personnalisé. Nous passons maintenant de cette description d'ensemble à sa mise en œuvre, en observant comment ProOWeb produit une application métier destinée à une organisation de commerce électronique.

Le chapitre suit donc le chemin emprunté par le framework : initialisation du projet, modélisation ou importation des processus, validation des spécifications enrichies, déploiement des contrats runtime et data, génération des artefacts backend/frontend et vérification des parcours obtenus. Cette progression permet de relier le modèle conceptuel du chapitre III au résultat produit dans l'application générée.

IV.2. Environnement de mise en œuvre

IV.2.1. Objectif de réalisation

La mise en œuvre de ProOWeb a consisté à rendre opérationnel le framework décrit au chapitre précédent. Les blocs conceptuels présentés dans l'architecture de la section III.3 sont ici traduits en composants réalisés : un noyau de génération, des mécanismes de validation, des contrats intermédiaires, des gabarits et un projet applicatif cible. Le framework reçoit des informations métier structurées, vérifie leur cohérence, construit des contrats intermédiaires et produit une base applicative exploitable. Cette base contient les éléments nécessaires pour exécuter un processus, afficher les tâches, appliquer les rôles, manipuler les données et suivre les fichiers générés.

IV.2.2. Choix technologiques

Le générateur a été séparé de l'application produite. Cette séparation reprend la distinction établie au chapitre III entre le noyau de génération et l'application web générée. ProOWeb s'exécute comme une application Node.js. Il fournit une interface de configuration, des services de gestion du workspace et des modules spécialisés dans la modélisation de processus : catalogue, validation de *spec-v1*, simulation, déploiement, génération des contrats runtime et gestion des fichiers produits.

L'application générée, quant à elle, repose sur Spring Boot pour le backend, React/Vite pour l'interface web et PostgreSQL comme base de données cible.

TABLE XVI – *Principaux choix de mise en œuvre de ProOWeb*

Éléments	Choix retenu	Rôle dans ProOWeb
Noyau du framework	Node.js / JavaScript	Lire le workspace, importer les modèles, valider *spec-v1*, construire les contrats et déclencher le déploiement.
Interface de configuration	Pages web ProOWeb	Initialiser le projet, accéder au tableau de bord et piloter certaines opérations de génération.
Backend généré	Spring Boot modulaire	Porter les modules système, organisation, sécurité, runtime de processus et ressources métier générées.
Frontend généré	React/Vite	Fournir les vues de travail, notamment le *Process Runtime Workbench* utilisé dans les tests.
Base de données cible	PostgreSQL avec Liquibase	Préparer la persistance et les scripts de migration nécessaires au projet produit.
Suivi des fichiers	Manifeste de génération	Identifier les fichiers générés et soutenir les régénérations contrôlées.
Vérification	Scripts et tests Cypress	Vérifier les parcours frontend liés aux processus déployés.

IV.2.3. Chaîne de génération

La figure 30 présente la chaîne de génération mise en œuvre dans ProOWeb. Elle reprend les étapes conceptuelles décrites dans la section III.5 : lecture des modèles, validation, construction d'une représentation intermédiaire, projection vers les artefacts et suivi de la génération. Elle montre ainsi le passage des modèles d'entrée vers les contrats intermédiaires, puis vers les artefacts applicatifs et le manifeste de traçabilité utilisé lors des régénérations.

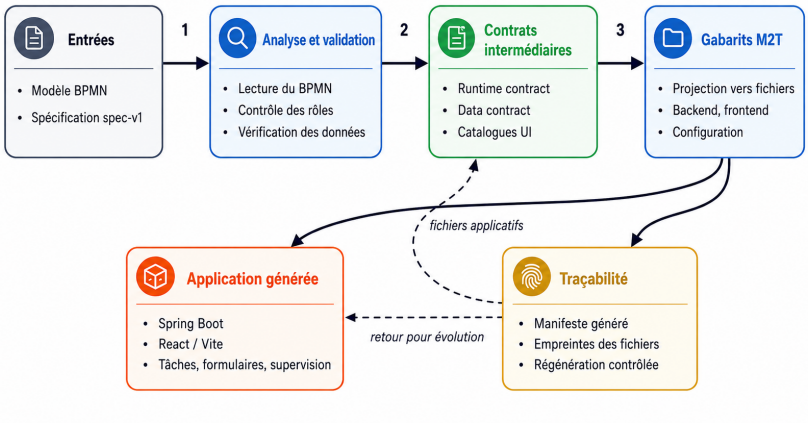


FIGURE 30 – *Chaîne de génération de ProOWeb*

IV.3. Cas d'étude organisationnel

IV.3.1. Présentation de l'organisation retenue

Le cas d'étude retenu concerne une organisation de commerce électronique. Cette organisation vend des produits à travers une boutique en ligne et s'appuie sur plusieurs pôles internes : prise en charge des commandes, gestion du stock, préparation logistique, service client et supervision commerciale. Son activité ne se résume donc pas à l'affichage d'un catalogue. Elle implique des décisions, des contrôles, des validations et des échanges entre acteurs avant qu'une commande soit clôturée ou qu'une réclamation soit traitée.

IV.3.2. Acteurs et responsabilités

Les acteurs retenus correspondent aux responsabilités courantes d'une organisation de commerce électronique.

TABLE XVII – *Acteurs du cas d'étude e-commerce*

Acteur	Responsabilité principale	Interaction avec l'application générée
Client	Passer une commande ou déposer une réclamation de produit	Renseigne les informations nécessaires et suit l'état de ses demandes.
Service commercial	Suivre les commandes et arbitrer les cas nécessitant une décision	Consulte les commandes, confirme certaines décisions et suit les exceptions.
Gestionnaire de stock	Vérifier les disponibilités, préparer les commandes et enregistrer les mouvements	Reçoit les tâches liées au stock, à la préparation et à la réception des produits.
Responsable achats	Valider les réapprovisionnements et suivre les demandes fournisseur	Examine les besoins de stock et autorise les commandes d'achat.
Service client	Qualifier les réclamations de produit et proposer une solution	Traite les demandes après-vente et renseigne les décisions prises.
Administrateur	Configurer les rôles, produits, catégories et paramètres généraux	Maintient les données de base utilisées par les processus générés.

IV.3.3. Processus retenus

Trois processus ont été retenus pour représenter le fonctionnement de cette organisation. La commande client couvre le parcours qui va de la soumission de la commande à sa préparation. Le réapprovisionnement du stock intervient lorsqu'un produit devient insuffisant ou lorsqu'un responsable anticipe une rupture. La réclamation de produit prend en charge les demandes formulées après une vente. Ces processus partagent des données et des acteurs, mais ils répondent à des objectifs opérationnels différents.

TABLE XVIII – *Processus métier retenus dans le cas d'étude*

Processus	Objectif métier	Acteurs clés	Résultat observé
P1 : commande client	Recevoir, vérifier, préparer et clôturer une commande passée sur la boutique en ligne	Client, service commercial, gestionnaire de stock	Commande confirmée, préparée, annulée ou clôturée.
P2 : réapprovisionnement du stock	Détecter un besoin de stock, le faire valider et enregistrer la réception des produits	Gestionnaire de stock, responsable achats	Stock réapprovisionné ou demande refusée.
P3 : réclamation de produit	Traiter une réclamation liée à une commande et proposer une solution adaptée	Client, service client, service commercial, gestionnaire de stock	Réclamation clôturée avec décision tracée.

IV.3.4. Scénario nominal et scénarios alternatifs

Dans le scénario nominal, le client soumet sa commande. Le service commercial en vérifie les informations et la déclare recevable. Le gestionnaire de stock confirme ensuite la disponibilité des produits, prépare la commande et enregistre cette préparation. Le service commercial confirme enfin que la commande est prête, puis le client consulte la confirmation.

Deux scénarios alternatifs complètent ce parcours. Lorsque la commande n'est pas recevable, elle est annulée et le client en est informé. Lorsqu'elle est recevable mais que le stock est insuffisant, elle est placée en attente et une notification est adressée au client. Ces trois issues servent de base à l'exemple détaillé présenté ci-après.

IV.4. Déroulement complet du processus de commande client

Cette section suit le chemin effectivement pris en charge par ProOWeb : initialiser le projet, modéliser le processus, compléter le BPMN avec *spec-v1*, valider les deux entrées, construire le modèle pivot puis générer le socle applicatif. Le processus de commande client sert de fil conducteur à cette démonstration.

IV.4.1. Initialisation du projet

La génération commence par l'initialisation du projet cible. Cette étape fixe l'identité de l'application à produire : nom du projet, espace de travail, package de base, options d'authentification, profils d'exécution et paramètres généraux. Dans le cas d'étude, cette initialisation a produit le workspace E-commerce Application, avec le package de base `com.prooweb.generated`, un backend Spring Boot, un frontend React et une base de données cible PostgreSQL. Le processus `commande-client` est ensuite choisi comme exemple de conception et d'exécution.

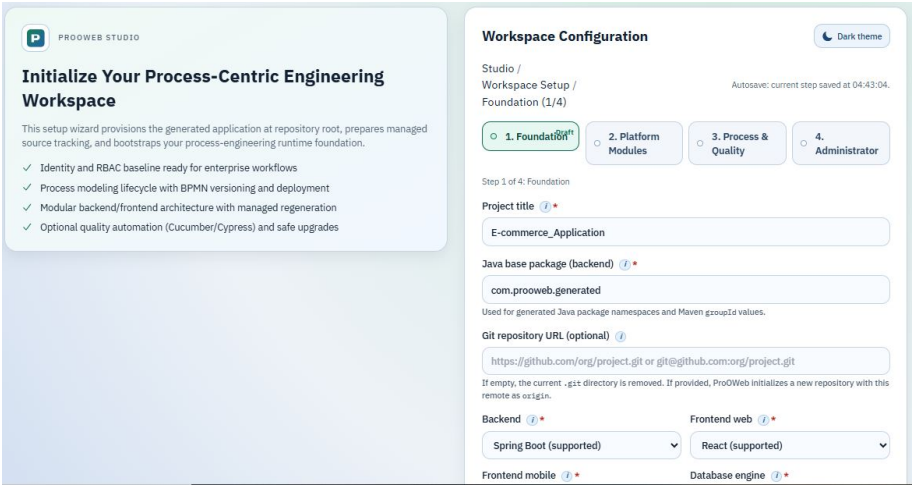


FIGURE 31 – *Initialisation du projet* `E-commerce_Application`

IV.4.2. Modélisation du processus

La commande client est modélisée dans le fichier `Commande_Client.bpmn`. Le processus réunit trois acteurs, représentés par les *lanes* Client, Service commercial et Gestionnaire de stock. Il comprend un événement de début, douze tâches utilisateur, deux passerelles exclusives et trois événements de fin. La première passerelle décide si la commande est recevable ; la seconde vérifie la disponibilité du stock. Les flux de séquence relient ces éléments et conduisent respectivement vers la confirmation, l'annulation ou la mise en attente. BPMN décrit ainsi l'ordre des activités, les responsabilités et les décisions du processus.

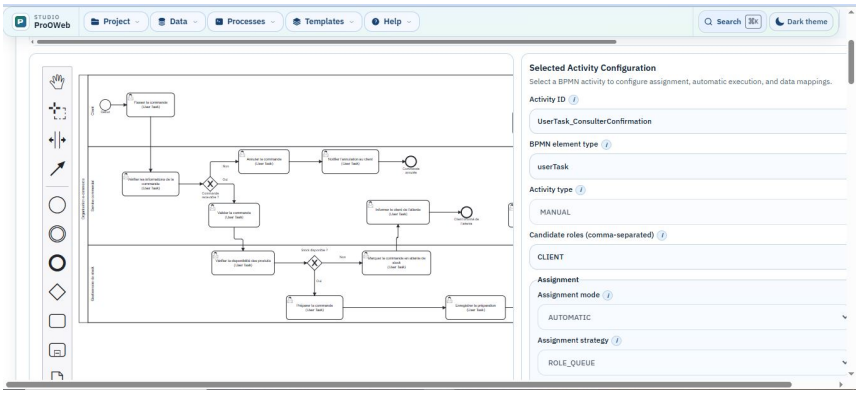


FIGURE 32 – *Modélisation du processus de commande client dans ProOWeb*

IV.4.3. Enrichissement du BPMN avec *spec-v1*

Le BPMN est complété par le fichier `commande-client.spec-v1.json`. Cet enrichissement suit le modèle grammatical défini par l'équation III.1 :

$$G_{\text{spec}} = (p, v, R, S, M, A, D, C).$$

La figure suivante illustre cette spécification pour l'activité « Passer la commande ».

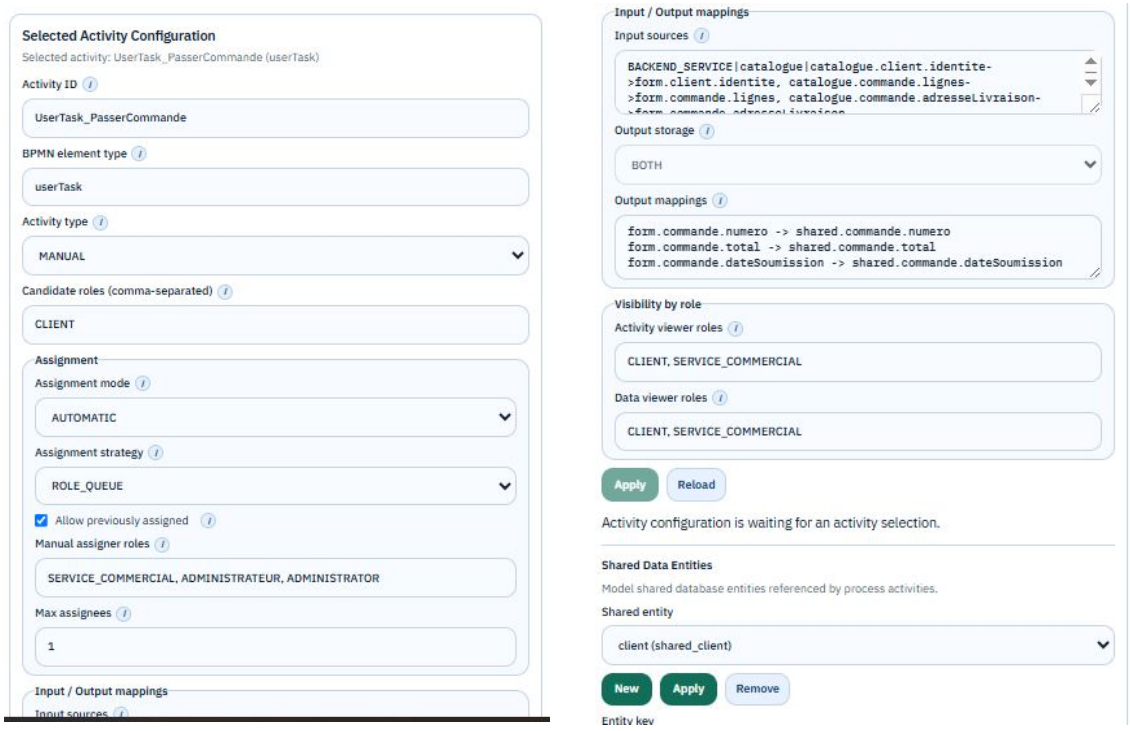


FIGURE 33 – *Spécification ajoutée à l'activité « Passer la commande »*

IV.4.4. Analyse et validation

ProOWeb analyse conjointement le BPMN et la spécification avant de poursuivre. Il vérifie que l'identifiant du processus correspond, que chaque activité enrichie existe dans le diagramme et que les codes de rôles respectent le format attendu. Il contrôle également les sources d'entrée, les sorties, les destinations des mappings, la présence des règles de visibilité et les références aux entités partagées. Une tâche manuelle doit notamment posséder au moins un rôle candidat. Lorsqu'une erreur bloquante est détectée, la construction des contrats est interrompue ; la génération ne part donc pas d'un couple de modèles incohérent.

Le listing IV.1 reproduit le rapport obtenu en appliquant le validateur de ProOWeb au bloc `specification` de `v1.json`, avec le BPMN déployé `v1.bpmn` et le mode strict. Les valeurs correspondent à la version 1 présente dans le workspace.

```
Action: specification/validate
Model: commande-client
Version: 1
Schema version: 1
Activities: 12
Manual activities: 12
Automatic activities: 0
Validation: valid
Errors: 0
Warnings: 0
```

Listing IV.1 – *Résultat de la validation BPMN–spec-v1 de la commande client*

Ce résultat atteste que les douze activités détectées dans le BPMN disposent d'une représentation acceptée par le validateur et qu'aucune erreur ni aucun avertissement n'est produit pour cette version. Une capture d'interface pourra montrer les sélecteurs `commande-client` et version 1, le bouton « Validate spec » et le message « Specification validation passed (0 errors, 0 warnings) ». Le contrôle de l'existence effective des rôles dans un annuaire organisationnel externe ne fait toutefois pas partie de ce résultat.

IV.4.5. Construction du modèle pivot

Après validation, la transformation M2M rassemble les informations des deux modèles dans le pivot ProOWeb. Le *ProcessModel* porte l'identité et la version du processus. Les *ActivityModel* représentent ses tâches ; les *RolePolicy* conservent l'affectation et la visibilité ; les *FormModel* décrivent les données attendues et produites. Le *RuntimeContract* organise les nœuds et les transitions, tandis que le *DataContract* formalise les mappings, le stockage et les données partagées. Les *ArtifactTrace* maintiennent enfin le lien entre les éléments sources et les artefacts

qui seront générés. Cette représentation intermédiaire évite aux générateurs de manipuler directement et séparément le BPMN et *spec-v1*.

Dans l'implémentation, le pivot est matérialisé par plusieurs contrats plutôt que par un fichier JSON unique. Le listing IV.2 rassemble un extrait du contrat runtime et du contrat data de la version 1. Une seule activité est conservée afin de faire apparaître la structure sans reproduire les fichiers complets.

```json
{
  "processModel": {
    "modelKey": "commande-client",
    "versionNumber": 1
  },
  "activityModel": {
    "activityId": "UserTask_AnnulerCommande",
    "candidateRoles": ["SERVICE_COMMERCIAL"],
    "assignment": {
      "mode": "AUTOMATIC",
      "strategy": "ROLE_QUEUE"
    },
    "formModel": {
      "outputs": ["commande.statut", "commande.motifAnnulation"]
    }
  },
  "runtimeContract": {
    "transition": {
      "id": "Flow_05",
      "source": "UserTask_AnnulerCommande",
      "target": "UserTask_NotifierAnnulationClient"
    }
  },
  "dataContract": {
    "storage": "BOTH",
    "mapping": {
      "from": "form.commande.statut",
      "to": "shared.commande.statut"
    }
  },
  "traceSimplifiee": {
    "sourceBpmn": "UserTask_AnnulerCommande",
    "contracts": ["v1.runtime.json", "v1.data.json"]
  }
}
```

Listing IV.2 – *Vue synthétique du modèle pivot de la commande client*

Le bloc `processModel` identifie la version du processus. Le bloc `activity Model` réunit l'activité, son rôle, son affectation et les sorties du formulaire. Le contrat runtime conserve la transition vers la notification du client, tandis que le

contrat de données projette le statut saisi vers la donnée partagée `commande`. Le dernier bloc est une trace de lecture simplifiée, reconstruite à partir des identifiants et des chemins versionnés ; il ne constitue pas un champ supplémentaire du contrat brut. Ces informations donnent aux générateurs une entrée stable pour produire les services, les formulaires et les règles de progression.

Le listing constitue ici la preuve textuelle attendue pour le pivot. Lors de la mise en forme finale, il pourra être complété par une capture montrant les fichiers `v1.runtime.json` et `v1.data.json` sous la même clé de processus, sans reproduire une seconde fois leur contenu.

IV.4.6. Génération des artefacts applicatifs

La transformation M2T projette le modèle pivot dans le workspace. Pour la commande client, ProOWeb conserve le BPMN dans `v1.bpmn` et la spécification normalisée dans `v1.json`. Il produit parallèlement les contrats runtime et data propres à cette version. Ces contrats alimentent les contrôleurs et services backend, les composants frontend, les formulaires d'activité, les règles d'accès, les fichiers de configuration et les catalogues communs. Le manifeste enregistre les chemins et les empreintes des fichiers gérés afin de soutenir les régénérations ultérieures. Le résultat constitue un socle applicatif organisé, destiné à être vérifié et complété par le développeur.

L'arborescence suivante reprend les artefacts effectivement observés dans le workspace. Elle est limitée aux fichiers utiles à la commande client ; les chemins intermédiaires ont été raccourcis, sans modifier les noms des fichiers.

```
E-commerce_Application/
|-- .prooweb-managed.json
`-- src/
    |-- backend/springboot/
    |   |-- prooweb-application/src/main/resources/
    |   |   |-- processes/
    |   |   |   |-- runtime-catalog.json
    |   |   |   |-- data-catalog.json
    |   |   |   `-- commande-client/
    |   |   |       |-- v1.bpmn
    |   |   |       |-- v1.json
    |   |   |       |-- v1.runtime.json
    |   |   |       `-- v1.data.json
    |   |   `-- db/changelog/changesets/
    |   |       |-- 900-process-shared-data.generated.yaml
    |   |       `-- 901-process-lane-roles.generated.yaml
    |   |-- system/system-domain/.../
    |   |   `-- CommandeClientProcessV1Spec.java
    |   |-- system/system-application/.../
```

```
    |   |   `-- ProcessRuntimeEngineService.java
    |   |-- system/system-infrastructure/.../
    |   |   |-- SharedCommandeEntity.java
    |   |   `-- SharedCommandeJpaRepository.java
    |   `-- gateway/.../ProcessRuntimeController.java
    `-- frontend/web/react/
        |-- src/modules/processes/
        |   |-- generatedProcessRegistry.js
        |   `-- commande-client/
        |       |-- ProcessCommandeClientV1Descriptor.js
        |       |-- ProcessCommandeClientV1RuntimeContract.js
        |       |-- ProcessCommandeClientV1DataContract.js
        |       `-- GeneratedCommandeClientV1ProcessPage.jsx
        `-- cypress/e2e/process-commande-client-v1.cy.js
```

Listing IV.3 – *Arborescence simplifiée des artefacts générés pour la commande client*

Cette arborescence montre que la transformation M2T relie les ressources versionnées, le runtime backend, la persistance des données partagées, les composants React et le scénario de test propre au processus. Une capture de l'explorateur de fichiers pourra compléter ce listing si elle rend visibles ces mêmes chemins.

IV.4.7. Fonctionnement dans l'application web générée

Dans l'application générée, l'utilisateur authentifié accède d'abord au catalogue des processus autorisés pour son rôle. Un client peut sélectionner la commande client, créer une instance et ouvrir le formulaire associé à la tâche « Passer la commande ». Après validation, les données sont transmises selon les mappings du contrat data et la tâche suivante devient disponible pour le service commercial. La progression se poursuit ensuite selon les décisions de recevabilité et de disponibilité définies dans le BPMN. Chaque acteur ne voit que les tâches et les données autorisées par les politiques issues de *spec-v1*, tandis que les rôles de suivi peuvent consulter l'état de l'instance.

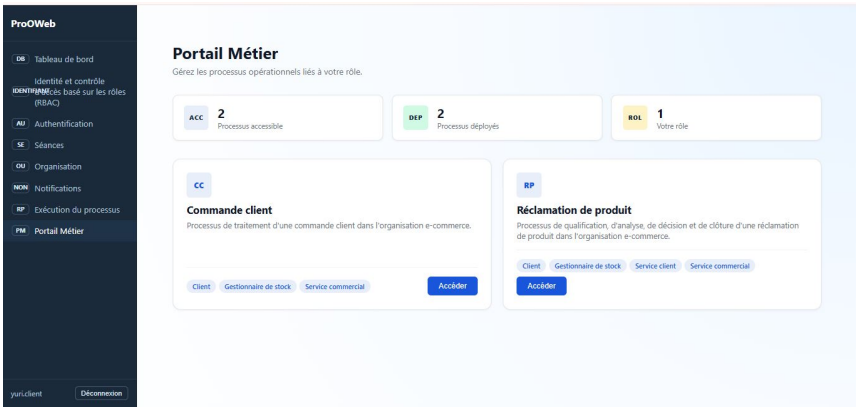


FIGURE 34 – *Accès à la liste des processus et démarrage de la commande client*

Cette interface présente les processus accessibles à l'utilisateur selon son rôle.

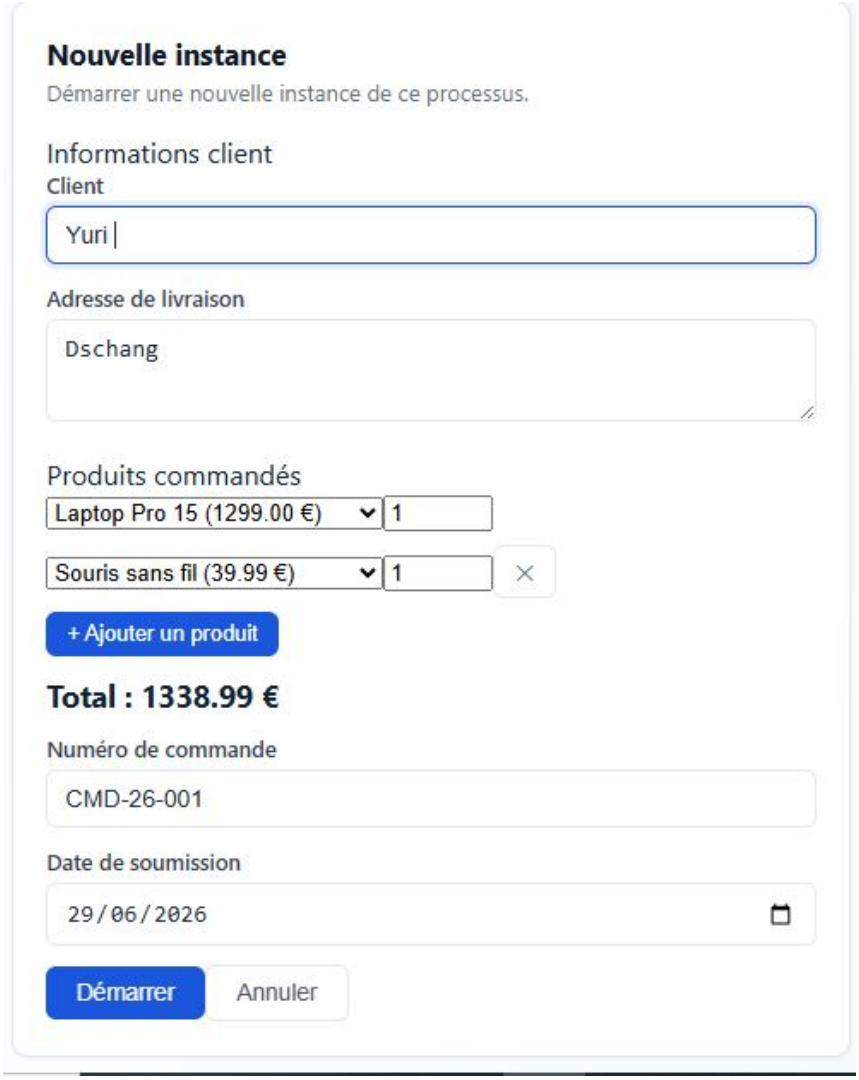


FIGURE 35 – *Formulaire associé à l'activité « Passer la commande »*

Le formulaire recueille les informations du client, les produits et les quantités avant le démarrage de l'instance. Il matérialise les entrées définies dans *spec-v1*.

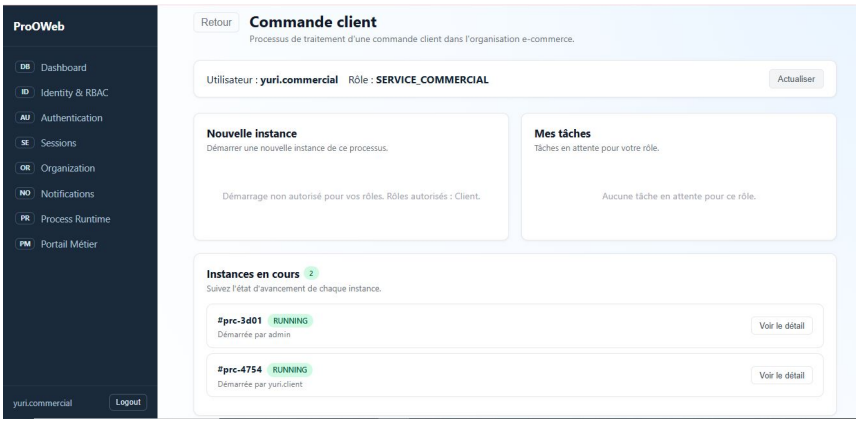


FIGURE 36 – *Transition vers l'activité de vérification affectée au Service commercial*

Cette vue présente les instances en cours et les tâches accessibles au Service commercial. Elle illustre l'application des droits de démarrage et d'accès associés au rôle.

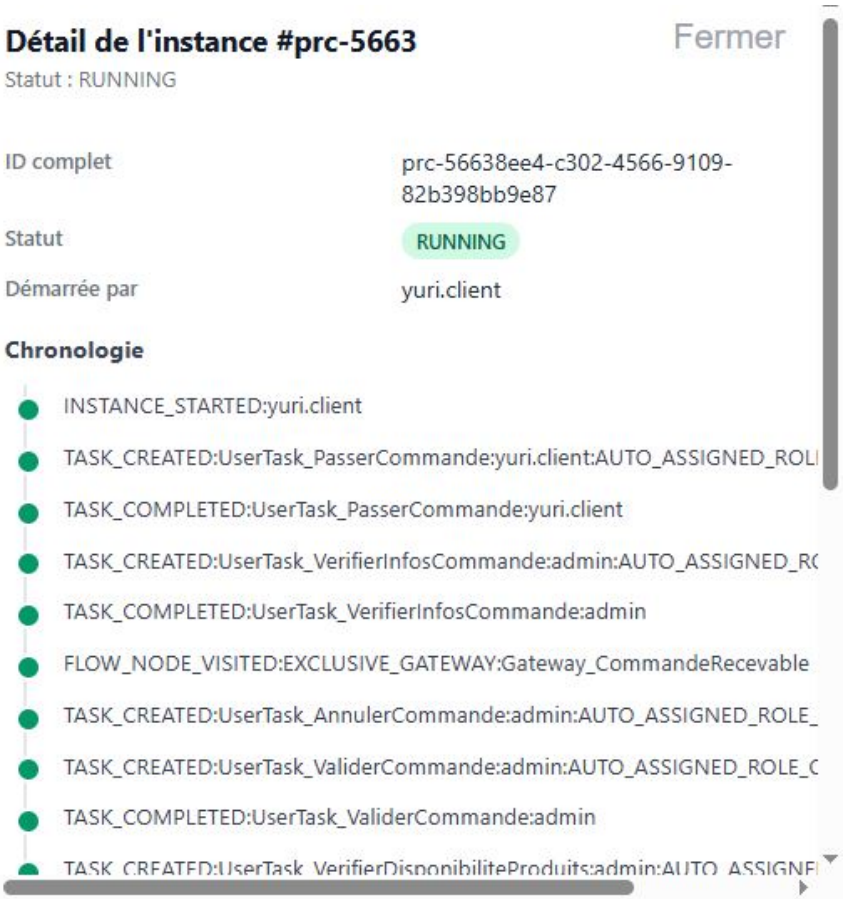


FIGURE 37 – *Suivi de l'instance de commande*

Le détail de l'instance restitue son état et la chronologie des tâches et passerelles traversées. Il rend observable la progression du processus exécuté par le runtime.

IV.5. Discussion et positionnement

Discussion générale

Le cas d'étude e-commerce montre que ProOWeb permet de passer de processus métier exprimés en BPMN à un socle applicatif web structuré. Pour chaque processus étudié, le modèle BPMN est complété par une spécification `spec-v1`, puis projeté dans des contrats `runtime` et `data` avant la génération des artefacts applicatifs. Le résultat ne se limite donc pas à une juxtaposition de fichiers : il matérialise une chaîne cohérente entre le processus métier, son enrichissement, le modèle pivot et le projet logiciel généré. Cette continuité constitue le principal résultat observé dans le périmètre du cas d'étude.

Forces de l'approche

La première force de ProOWeb réside dans la continuité qu'il établit entre la modélisation métier et la réalisation logicielle. BPMN conserve la description du déroulement du processus, tandis que `spec-v1` apporte les rôles, les données, les règles d'affectation, les mappings et les contraintes de visibilité nécessaires à la génération. La validation croisée limite la propagation d'incohérences avant la transformation. Le modèle pivot stabilise ensuite les informations validées dans une représentation commune, exploitée par les transformations `M2M` et `M2T`. Les contrats intermédiaires, les traces et le manifeste maintiennent enfin le lien entre les modèles d'entrée et les fichiers produits. Cette organisation rend le socle généré plus lisible, traçable et extensible, tout en laissant au développeur la possibilité de l'adapter aux besoins du projet.

Limites observées et positionnement

Ces résultats doivent néanmoins être interprétés dans les limites de l'évaluation menée. Le cas d'étude reste centré sur une organisation e-commerce et ne suffit pas à établir la généralisation de l'approche à tous les domaines métier. Les scénarios Cypress vérifient principalement des parcours frontend dans un environnement contrôlé ; l'utilisation d'interceptions d'API ne démontre pas encore l'intégration complète avec le backend réel, PostgreSQL, les mécanismes de sécurité et la persistance. La qualité du projet généré dépend également de la précision du BPMN et de la spécification `spec-v1`. Enfin, la préservation du code personnalisé lors de régénérations successives devra être éprouvée sur des évolutions plus importantes. ProOWeb ne se positionne donc ni comme un BPMS classique, ni comme une pla-

teforme low-code généraliste, ni comme un substitut au développeur. Il constitue un framework d'ingénierie dirigée par les modèles destiné à réduire l'écart entre le modèle métier et le projet logiciel, en fournissant une base applicative fonctionnelle, structurée, traçable et extensible que le développeur peut vérifier, adapter et compléter.

IV.6. Conclusion

Ce chapitre a présenté la mise en œuvre de ProOWeb et son application à un cas d'étude centré sur une organisation de commerce électronique. Il a montré comment les modèles BPMN enrichis sont importés, validés, transformés en contrats, puis déployés dans une application web métier générée.

Le cas pratique a permis d'observer ProOWeb sur trois processus appartenant au même contexte : commande client, réapprovisionnement du stock et réclamation de produit. L'évaluation confirme la production d'un socle applicatif commun et traçable. Elle ne valide toutefois qu'une partie de son comportement : la qualité des modèles d'entrée reste déterminante, l'enrichissement demande un effort de spécification et les tests frontend couvrent seulement des préfixes nominaux avec API simulée. Des essais d'intégration et de régénération sont donc nécessaires avant de conclure sur un usage en production.

CONCLUSION GÉNÉRALE

SOMMAIRE

Bilan du travail 80
Quelques perspectives 81

Bilan du travail

Ce mémoire est parti d'un décalage concret : BPMN décrit avec précision le déroulement d'un processus, mais ne porte pas, à lui seul, toutes les décisions nécessaires à une application web métier. Les formulaires, les données persistées, les affectations, les règles de visibilité, la sécurité et la cohabitation entre code généré et code personnalisé restent à préciser. La problématique consistait donc à produire un socle applicatif web à partir de modèles BPMN enrichis tout en maintenant la cohérence entre ces dimensions et les évolutions futures du projet.

ProOWeb répond à ce problème par une chaîne d'ingénierie explicite. Le BPMN reste la référence du workflow ; la spécification *spec-v1* décrit les informations applicatives absentes du diagramme. Leur validation croisée alimente un modèle pivot, matérialisé par des contrats runtime et data. Des transformations modèle-à-modèle puis modèle-à-texte projettent ces contrats vers un backend Spring Boot, un frontend React, des ressources de données, des règles d'accès et des composants d'exécution. Enfin, les clés de processus, les versions, les contrats et le manifeste maintiennent un lien observable entre les modèles et les fichiers produits.

Les principales contributions de ce travail sont les suivantes :

— une spécification enrichie, formalisée par une grammaire et des contraintes de validité, qui complète BPMN sans transformer le diagramme en description technique de toute l'application ;

— une architecture séparant les sources, les validations, les contrats intermédiaires, les générateurs et le projet cible ;

— des algorithmes explicites de validation, de construction des contrats, de génération et de régénération couvrant le runtime, le backend, le frontend, les données et la sécurité ;
— un dispositif de traçabilité et de régénération fondé notamment sur un manifeste, des empreintes et des points d'extension ;
— une mise à l'épreuve sur trois processus liés d'une organisation de commerce électronique.

Le cas d'étude fournit des résultats vérifiables, mais circonscrits. Les trois processus disposent chacun d'un BPMN, d'une spécification normalisée, d'un contrat runtime et d'un contrat data dans un même workspace. Des catalogues globaux, des classes Java, des composants React, des scénarios Cucumber et trois tests Cypress complètent ces ressources. Cette observation confirme la production d'un socle multi-processus cohérent et traçable. Elle ne démontre pas encore toutes les propriétés d'une application en production : les scénarios Cypress simulent le backend et ne couvrent que deux ou trois activités par processus ; la persistance réelle, la sécurité, la charge, les interactions inter-processus et la régénération après personnalisation demandent des essais supplémentaires.

Le résultat doit donc être formulé avec mesure. ProOWeb ne remplace ni l'analyse métier ni le développeur. Il réduit la distance entre le processus et le projet logiciel en produisant un socle fonctionnel, traçable et extensible. L'équipe de développement reste responsable des règles non modélisées, des intégrations externes, de l'expérience utilisateur et de la validation dans son environnement.

Quelques perspectives

Les prolongements prioritaires découlent directement de ces limites :

— exécuter des campagnes d'intégration sans API simulée, avec PostgreSQL, authentification, autorisations et services externes ;
— tester systématiquement les branches BPMN, les transitions, les rôles, les formulaires et les règles de visibilité ;
— évaluer la régénération sur des fichiers effectivement personnalisés et mesurer les conflits, sauvegardes et migrations produits ;
— assister la construction conjointe du BPMN et de *spec-v1*, notamment par des diagnostics compréhensibles par le concepteur ;
— reproduire l'étude sur d'autres domaines, des modèles plus complexes et d'autres piles techniques.

En définitive, ce travail montre l'intérêt d'une continuité explicite entre modèle métier, spécification enrichie, contrats intermédiaires, artefacts générés et évolution contrôlée. C'est dans cette continuité, plus que dans la seule quantité de code produite, que se situe la contribution de ProOWeb.

BIBLIOGRAPHIE

[1] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*. Springer, Berlin, Heidelberg, 2 edition, 2018.

[2] Mathias Weske. *Business Process Management : Concepts, Languages, Architectures*. Springer, Berlin, Heidelberg, 2 edition, 2012.

[3] Object Management Group. Business Process Model and Notation (BPMN), Version 2.0.2. Technical report, Object Management Group, 2013. Consulté le 22 mai 2026.

[4] Object Management Group. Model Driven Architecture Guide, Revision 2.0. Technical report, Object Management Group, 2014. Guide officiel de la Model Driven Architecture.

[5] Jean-Marc Jézéquel, Benoît Combemale, and Didier Vojtisek. *Ingénierie dirigée par les modèles : des concepts à la pratique*. Ellipses, Paris, 2012.

[6] Eduardo Díaz, José Ignacio Panach, Silvia Rueda, and Oscar Pastor. Towards a method to generate GUI prototypes from BPMN. In *2018 12th International Conference on Research Challenges in Information Science (RCIS)*, pages 1–12. IEEE, 2018.

[7] Abir Sajji, Yassine Rhazali, and Youssef Hadi. A methodology for transforming bpmn to ifml into mda. *Bulletin of Electrical Engineering and Informatics*, 11(5) :2773–2782, 2022.

[8] Abir Sajji, Yassine Rhazali, and Youssef Hadi. Graphical user interfaces generation from bpmn via ifml up to psm level. *International Journal of Advanced Computer Science and Applications*, 14(2) :419–431, 2023.

[9] Imke Drave, Judith Michael, Erik Müller, Bernhard Rumpe, and Simon Varga. Model-driven engineering of process-aware information systems. *SN Computer Science*, 3(479), 2022.

[10] Michael Hammer. Reengineering work : Don't automate, obliterate. *Harvard Business Review*, 1990.

[11] Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, and Mathias Weske. *Business Process Management : A Survey*, volume 2678 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2003.

[12] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. *Fundamentals of Business Process Management*, volume 1. Springer, Berlin, 2013.

[13] Bonitasoft. L'essentiel du bpmn 2 : Comment réconcilier les besoins de la modélisation métier et informatique pour la gestion des processus métiers. Guide technique, Bonitasoft, s.d.

[14] Peter Pichler, Barbara Weber, Stefan Zugal, Jakob Pinggera, Jan Mendling, and Hajo A. Reijers. Imperative versus declarative process modeling languages : An empirical investigation. In *Business Process Management Workshops*, volume 99 of *Lecture Notes in Business Information Processing*, pages 383–394. Springer, 2011.

[15] Mark von Rosing, Stephen White, Fred Cummins, and Henk de Man. Business process model and notation – bpmn. In Mark von Rosing, Henrik von Scheel, and August-Wilhelm Scheer, editors, *The Complete Business Process Handbook : Body of Knowledge from Process Modeling to BPM, Volume I*, pages 429–453. Morgan Kaufmann/Elsevier, Waltham, MA, 2015.

[16] Samir Meghzili. Ingénierie des logiciels : chapitre ingénierie dirigée par les modèles, 2022. Support de cours.

[17] Object Management Group. Meta object facility (mof). https://www.omg.org/spec/MOF/, s.d. Spécification officielle OMG.

[18] Object Management Group. Mof query/view/transformation (qvt). https://www.omg.org/spec/QVT/, s.d. Spécification officielle OMG.

[19] Ian Jacobs and Norman Walsh. Architecture of the world wide web, volume one. Technical report, World Wide Web Consortium, 2004. Consulté le 22 mai 2026.

[20] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Semantics. Technical Report RFC 9110, Internet Engineering Task Force, 2022. Consulté le 22 mai 2026.

[21] MDN Web Docs. How the web works. https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works, 2026. Consulté le 22 mai 2026.

[22] Avraham Leff and James T. Rayfield. Web-application development using the model/view/controller design pattern. In *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*, pages 118–127. IEEE, 2001.

[23] MDN Web Docs. Spa : Single-page application. https://developer.mozilla.org/en-US/docs/Glossary/SPA, 2026. Consulte le 09 juin 2026.

[24] Roy Thomas Fielding. *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.

[25] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2) :38–47, February 1996.

[26] Camunda. Camunda Modeler and BPMN Process Orchestration Documentation. https://docs.camunda.io/docs/components/modeler/about-modeler/, 2026. Documentation officielle consultée en 2026.

[27] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical report, Keele University and Durham University, 2007.

[28] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, et al. The prisma 2020 statement : an updated guideline for reporting systematic reviews. *BMJ*, 372 :n71, 2021.

[29] Wil M. P. van der Aalst. Business process management : a comprehensive survey. *ISRN Software Engineering*, 2013 :507984, 2013.

[30] Flowable. Flowable Open Source BPMN Documentation. https://www.flowable.com/open-source/docs/bpmn/ch02-GettingStarted/, 2026. Documentation officielle consultée en 2026.

[31] jBPM. jBPM Documentation : BPMN 2.0 Workflow Engine. https://www.jbpm.org/learn/documentation.html, 2026. Documentation officielle consultée en 2026.

[32] Activiti. Activiti BPMN Workflow Engine Documentation. https://www.activiti.org/userguide/, 2026. Documentation officielle consultée en 2026.

[33] Eike Schäffer, Marvin Schobert, Tobias Reichenstein, Andreas Selmaier, Volker Stiehl, Matthias Herhoffer, Michael Mala, and Jörg Franke. Reference architecture and agile development method for a process-driven web platform based on the BPMN-standard and process engines. *Procedia CIRP*, 103 :146–151, 2021.

[34] Zef Hemel, Ruben Verhaaf, and Eelco Visser. Webworkflow : An object-oriented workflow modeling language for web applications. In *Proceedings of the 11th International Conference on Model Driven Engineering Languages and Systems (MODELS 2008)*, volume 5301 of *Lecture Notes in Computer Science*, 2008.

[35] Ulisses Telemaco Neto, Toacy Oliveira, Raquel Pillat, Paulo Alencar, Don Cowan, and Glaucia Melo. Akip process automation platform : A framework for the development of process-aware web applications. In *Proceedings of the International Conference on Enterprise Information Systems*, 2022.

[36] Abir Sajji, Yassine Rhazali, and Youssef Hadi. From interaction flow modeling language to symfony : an automated model transformation methodology. *Bulletin of Electrical Engineering and Informatics*, 14(5) :3765–3777, 2025.

[37] Naziha Laaz, Nassim Kharmoum, and Samir Mbarki. Combining domain ontologies and bpmn models at the cim level to generate ifml models. In *Procedia Computer Science*, volume 170, pages 851–856, 2020.

[38] Daniel Pietruck and Ulrike Steffens. Generating REST APIs for workflow engines with process and data models. In *Proceedings of the 12th Central European Workshop on Services and their Composition*, volume 2825 of *CEUR Workshop Proceedings*, 2020.

[39] Jean Pierre Alfonso Hoyos and Felipe Restrepo-Calle. Automatic source code generation for web-based process-oriented information systems. In *Proceedings of ENASE*, 2017.

[40] Burak Uyanik and Ahmet Sayar. Developing web-based process management with automatic code generation. *Applied Sciences*, 13(21) :11737, 2023.

[41] Eugene Syriani, Lechanceux Luhunu, and Houari Sahraoui. Systematic mapping study of template-based code generation. *Computer Languages, Systems & Structures*, 52 :43–62, 2018.

[42] Arianna Fedeli, Fabrizio Fornari, Andrea Polini, Barbara Re, Victoria Torres, and Pedro Valderas. Flobp : a model-driven approach for developing and executing iot-enhanced business processes. *Software and Systems Modeling*, 23 :1217–1246, 2024.

[43] Rubén Salado-Cid, Antonio Vallecillo, Kamran Munir, and José Raúl Romero. Swel : A domain-specific language for modeling data-intensive workflows. *Business & Information Systems Engineering*, 66 :137–160, 2024.

[44] Timo Greifenberg, Katrin Hölldobler, Carsten Kolassa, Markus Look, Pedram Mir Seyed Nazari, Klaus Müller, Antonio Navarro Pérez, Dimitri Plotnikov, David Reiss, Andreas Roth, Bernhard Rumpe, Martin Schindler, and Andreas Wortmann. A comparison of mechanisms for integrating handwritten and generated code for object-oriented programming languages. In *Proceedings of the 3rd International Conference on Model-Driven Engineering and Software Development*, pages 74–85, 2015.

[45] Khandoker Rahad, Omar Badreddin, and Syed Mohammad Reza. The human in model-driven engineering loop : A case study on integrating handwritten code in model-driven engineering repositories. *Software : Practice and Experience*, 51(6) :1308–1321, 2021.

[46] Bojan Dragaš, Nikola Todorović, Teodora Rajačić, and Goran Milosavljević. SeamlessMDD : Framework for seamless integration of generated and handwritten code. In *Web Engineering – 24th International Conference, ICWE 2024*, pages 163–177. Springer, 2024.

[47] Kristaps Rokis and Marite Kirikova. Exploring low-code development : A comprehensive literature review. *Complex Systems Informatics and Modeling Quarterly*, 36 :68–86, 2023.

[48] Małgorzata Pańkowska. Low code development cycle investigation, 2024.

[49] Andrea Delgado, Daniel Calegari, Ana Arrigoni, Alejandro Zunino, and Rodrigo Sosa. Model-driven management of BPMN-based business process families. *Software and Systems Modeling*, 2022.

[50] Alessio Bucaioni, Antonio Cicchetti, and Federico Ciccozzi. Modelling in low-code development : a multi-vocal systematic review. *Software and Systems Modeling*, 21 :1959–1981, 2022.

[51] Davide Di Ruscio, Dimitris Kolovos, Juan de Lara, Alfonso Pierantonio, Massimo Tisi, and Manuel Wimmer. Low-code development and model-driven engineering : Two sides of the same coin ? *Software and Systems Modeling*, 21 :437–446, 2022.

[52] Mariën R. Krouwel, Martin Op 't Land, and Henderik A. Proper. From enterprise models to low-code applications : mapping DEMO to mendix ; illustrated in the social housing domain. *Software and Systems Modeling*, 23(4) :837–864, 2024.

[53] Francisco Martínez-Lasaca, Pablo Díez, Esther Guerra, and Juan de Lara. A model and workflow-driven approach for engineering domain-specific low-code platforms and applications. *Software and Systems Modeling*, 25 :579–614, 2026.

[54] Brice Arléon Zemtsop Ndadji. Vers une approche basée sur l'ingénierie dirigée par les modèles, de gestion des workflows dans une application web. Mémoire de master en informatique, Université de Dschang, Dschang, Cameroun, 2022.

[55] Marionne Daniela Nguedia Momo. Vers une approche grammaticale de gestion des processus opérationnels dans un réseau IoT. Mémoire de master en informatique, Université de Dschang, Dschang, Cameroun, 2022.